

Computer Vision and Machine Learning for Scientific Visual Data Analytics

Lecture 5: Segmentation and Detection



Shu Kong
shu.kong@oist.jp



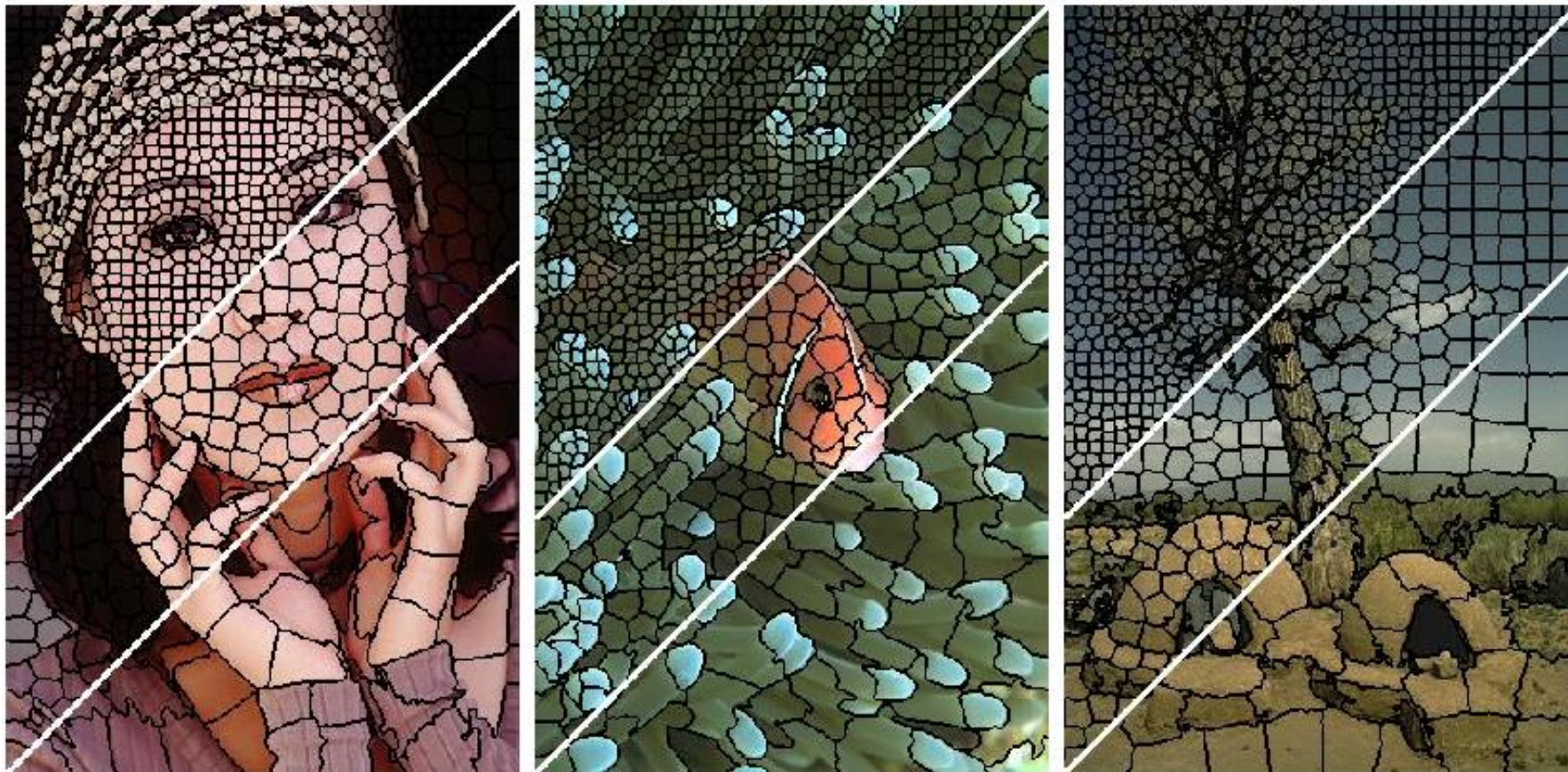
Roadmap

Teaching materials are available: <https://github.com/aimerykong/OIST-mini-course-CVML>

Lecture	Topics	Hands-on experience
1. Intro	CV/ML, deep learning, artificial intelligence	Configure programming environment using Google Drive and Colab; implement basic scripts in Python
2. Basics of CV/ML	training, validation, testing, features, metrics, nearest neighbor	Implement a machine learning pipeline for image classification
3. Classification	Logistic Regression, gradient descent	Train a linear classifier for pollen species recognition
4. Deep Learning	Multi-Layer Perceptron, feature learning, backpropagation, neural networks	Train a convolutional neural network (CNN) for pollen species recognition
5. Segmentation and Detection	semantic and instance segmentation, U-Net,	Train a neural network for pollen grain detection/segmentation
6. Foundation Models	CLIP , Segment Anything Model , DINOv2 , GroundingDINO , finetuning	Finetune a pretrained model for image classification and segmentation
7. Advanced Topics	open-set recognition, novel species recognition, phylogentic reconstruction via clustering	Implement algorithms to recognize unknown/anomalous/novel observations
8. Brainstorm & Talks	Volunteer to present	n/a

The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing



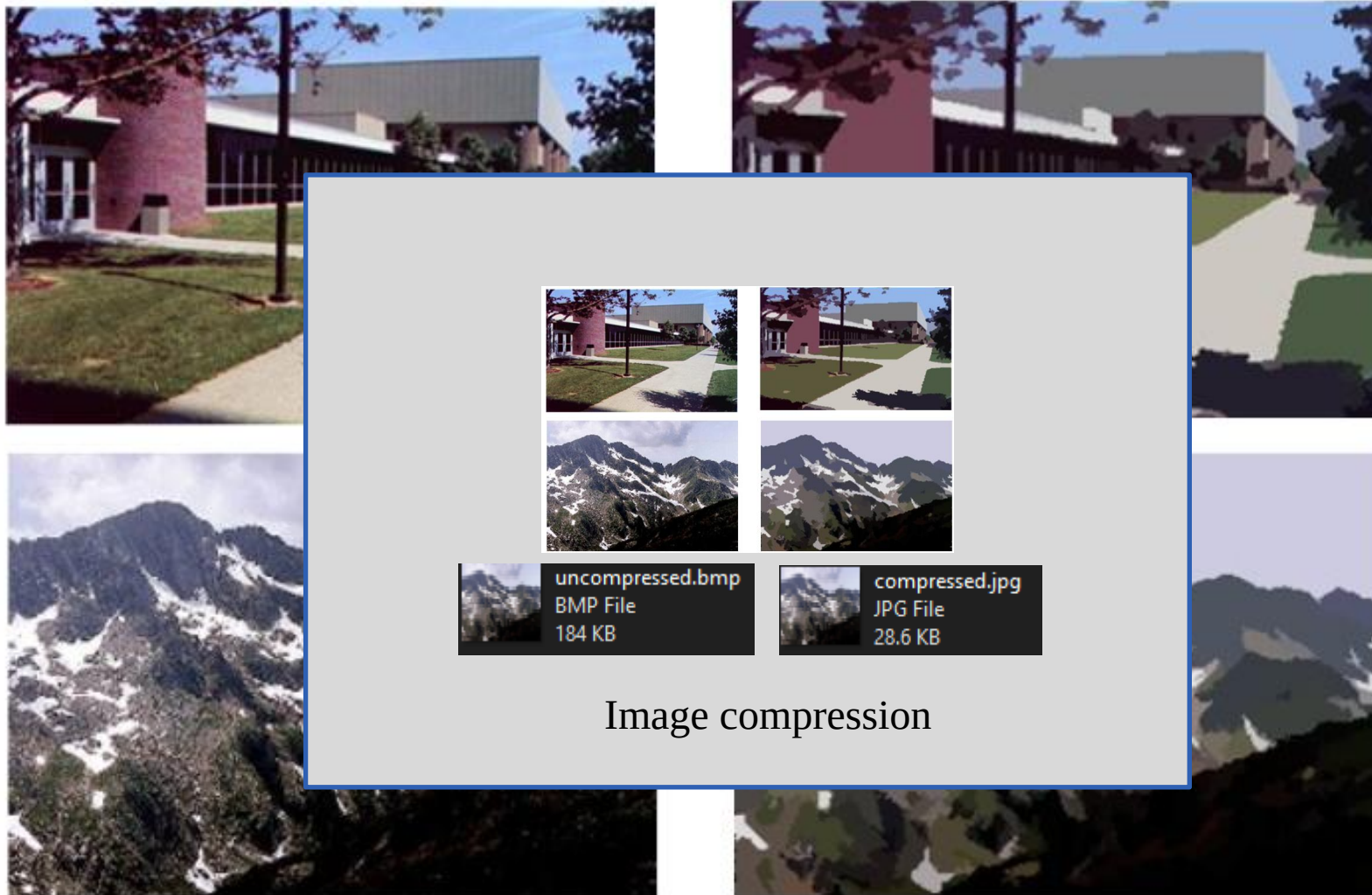
The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing



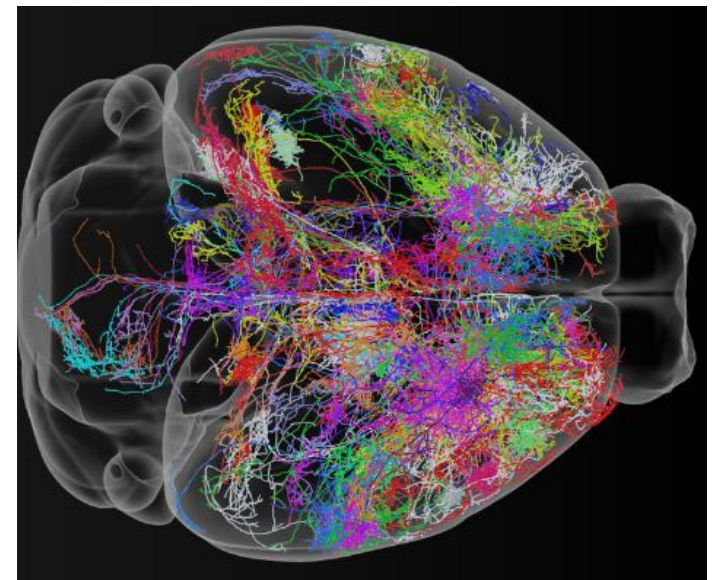
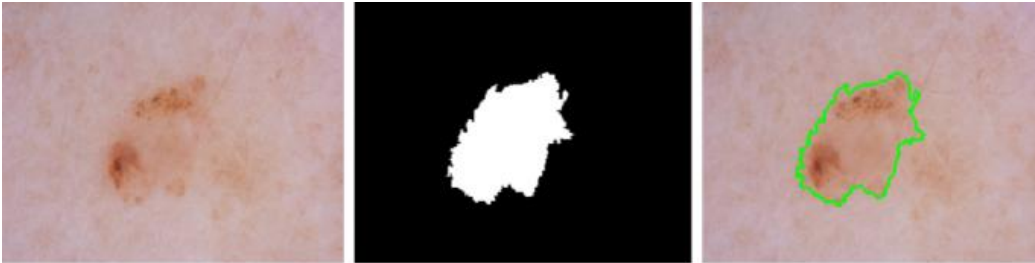
The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing



The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing
- Separate image into coherent “objects”

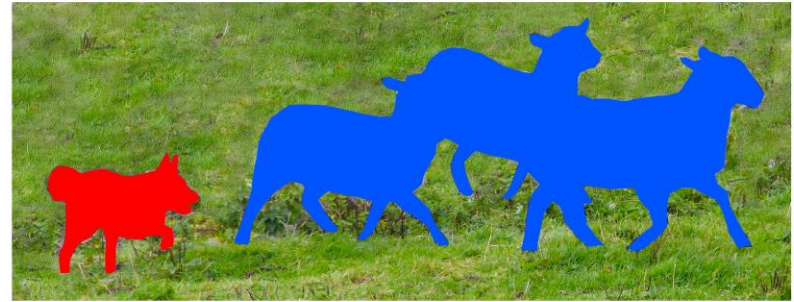


The goals of segmentation

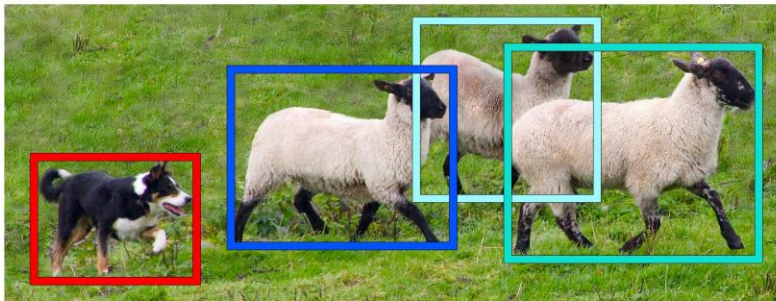
- Group together similar-looking pixels for efficiency of further processing
- Separate image into coherent “objects”



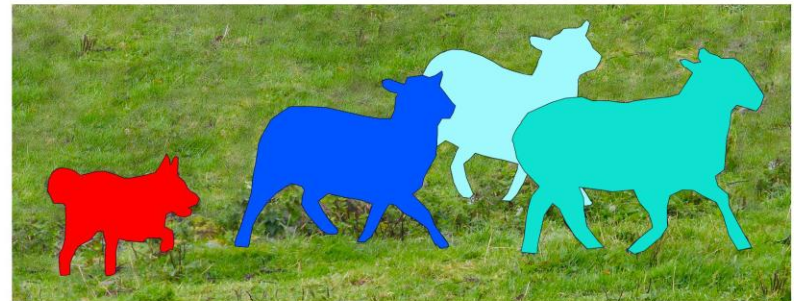
Image Recognition



Semantic Segmentation



Object Detection

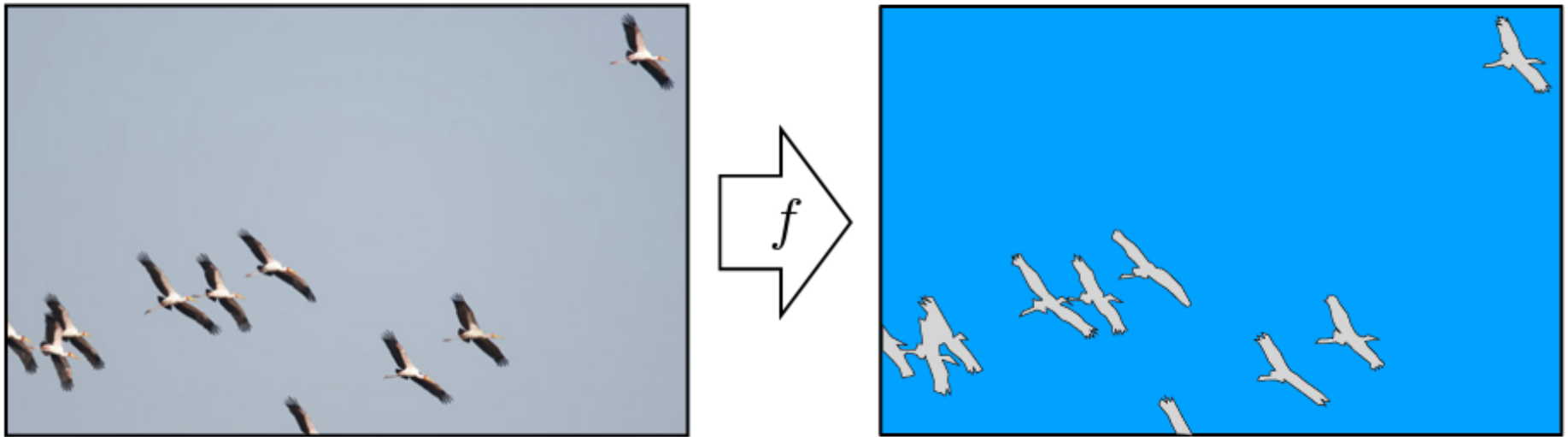


Instance Segmentation

Outline

- **Deep Learning for Semantic Segmentation**
- Deep Learning for Instance Segmentation

Semantic segmentation



(Colors represent categories)

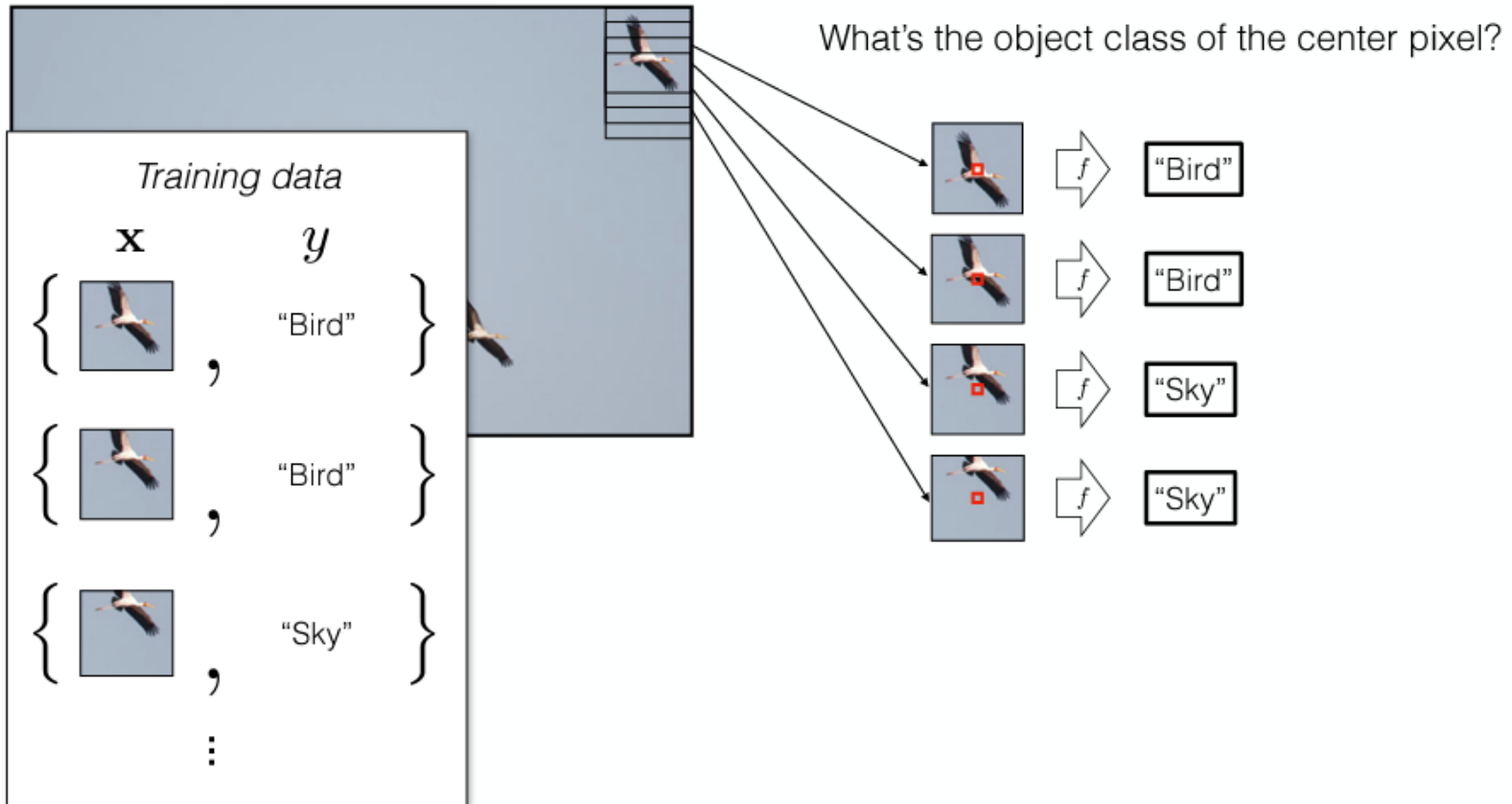
General technique: ³⁵predict something at every pixel!

Deep Learning for Semantic Segmentation

- Idea 1: independently classify windows

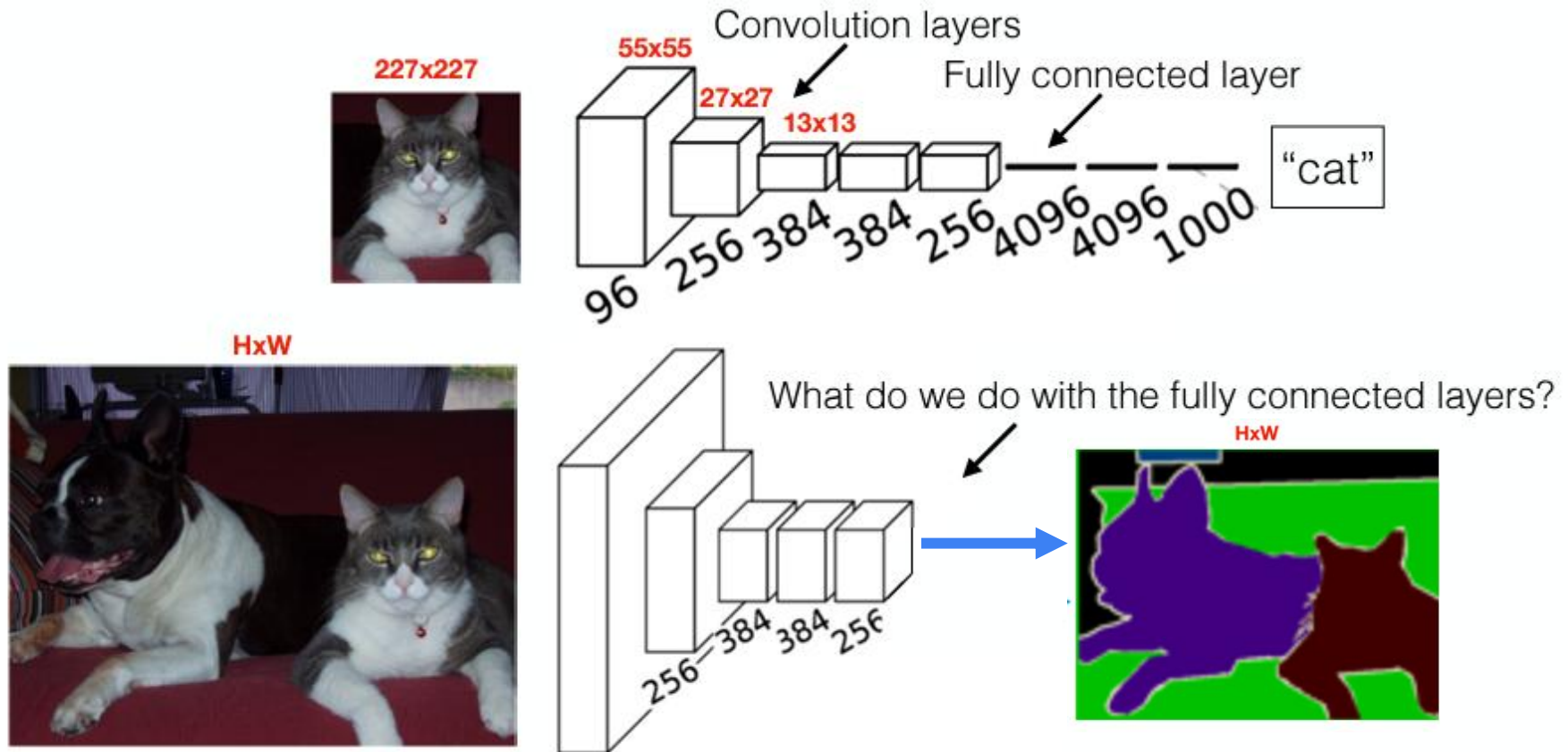
Deep Learning for Semantic Segmentation

- Idea 1: independently classify windows



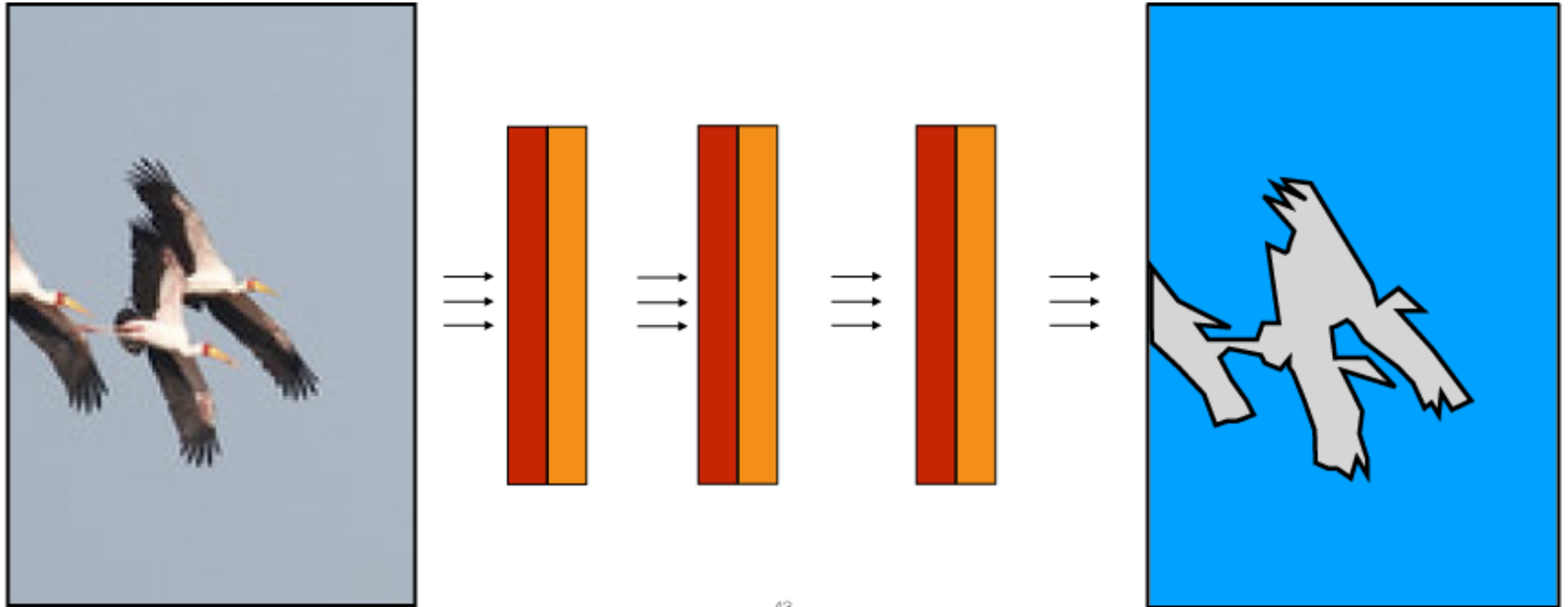
Deep Learning for Semantic Segmentation

- Idea 1: independently classify windows
- Idea 2: convolutionally classify pixels



Deep Learning for Semantic Segmentation

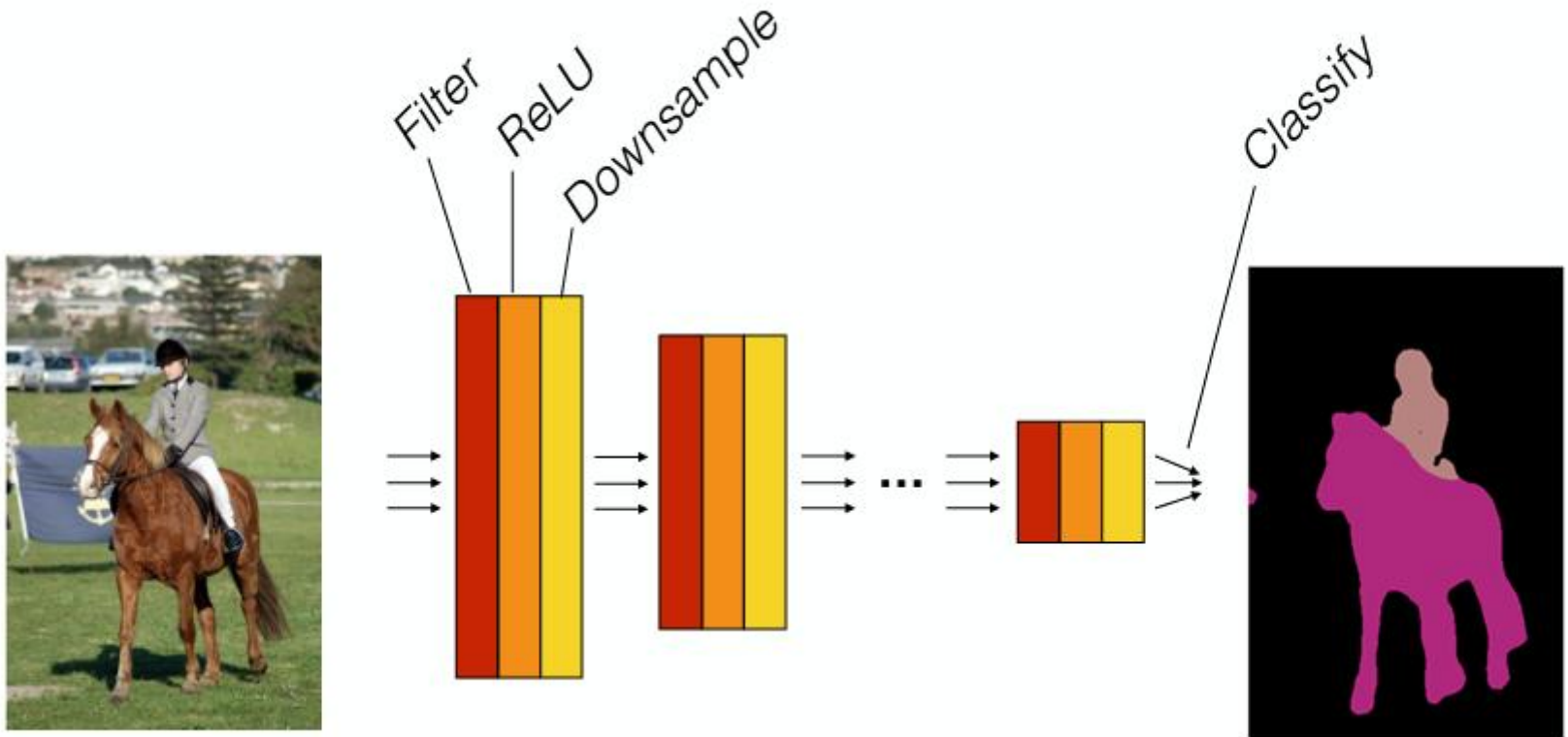
What if we remove subsampling layers?



43

- Small receptive fields
- Computationally expensive

Deep Learning for Semantic Segmentation

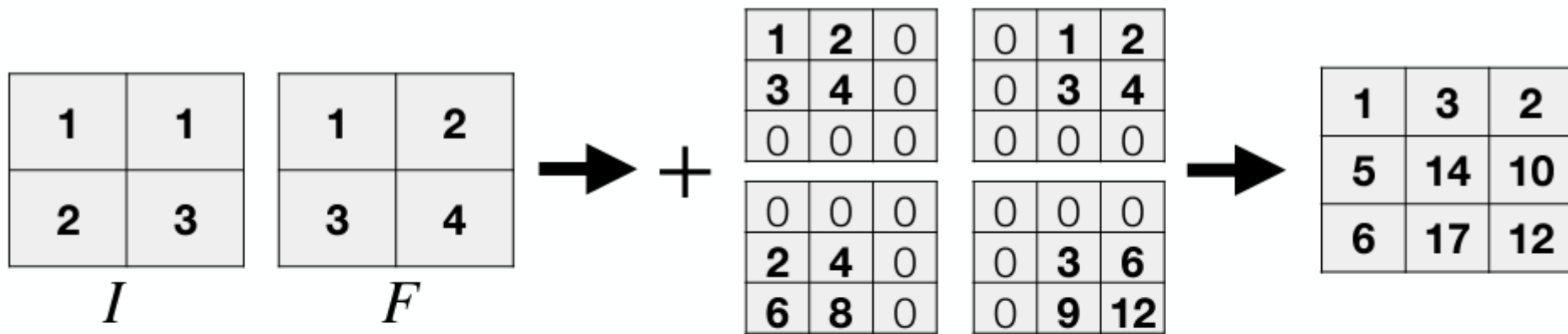


Upsampling



- Often using nearest-neighbor upsampling
- Can also use interpolation.
- Produces fewer “checkerboard” artifacts

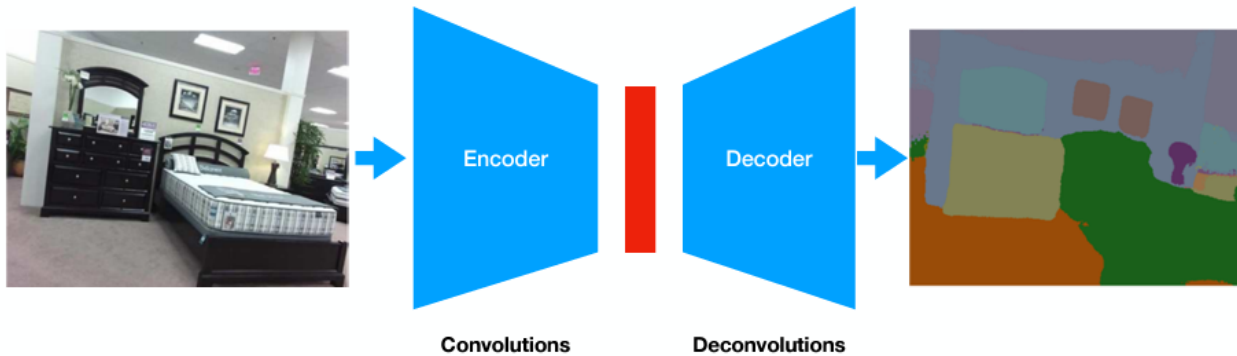
Transposed convolution



- Weight the filter by the image coefficient and sum.
- Also sometimes called “upconvolution” or “deconvolution”.

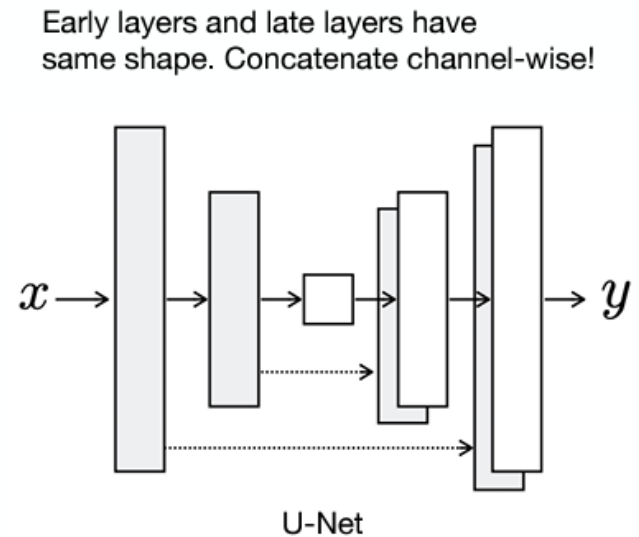
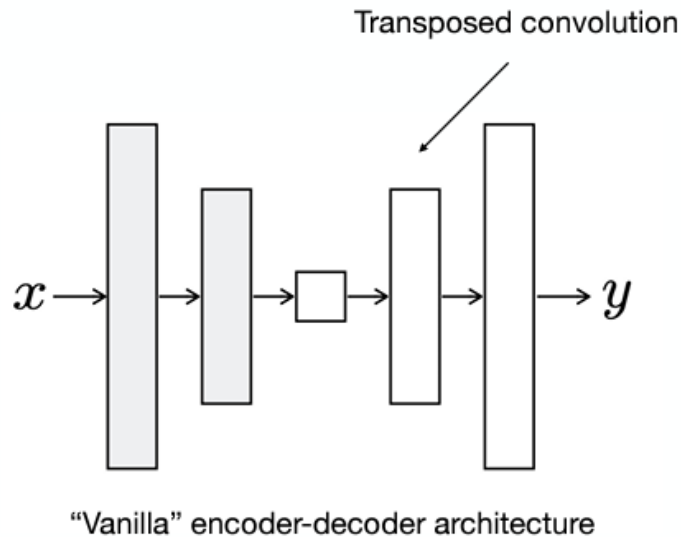
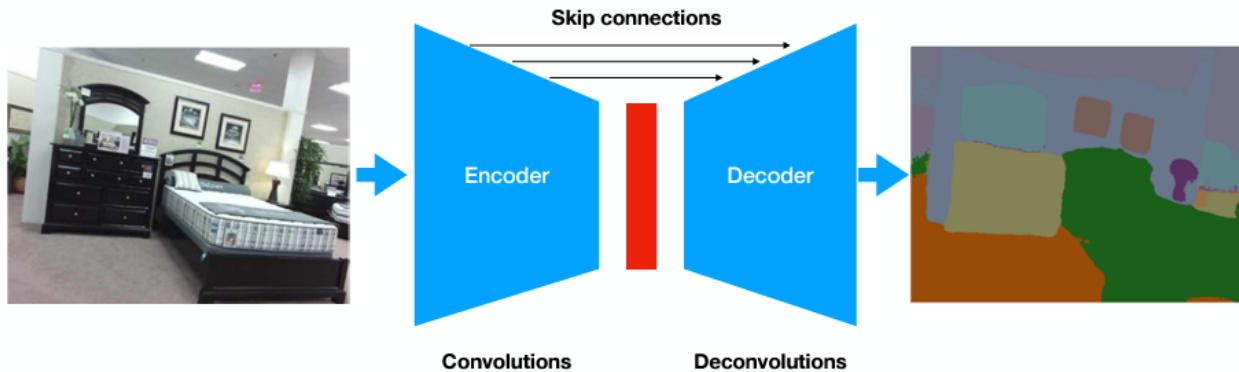
Deep Learning for Semantic Segmentation

Encoder-decoder architectures



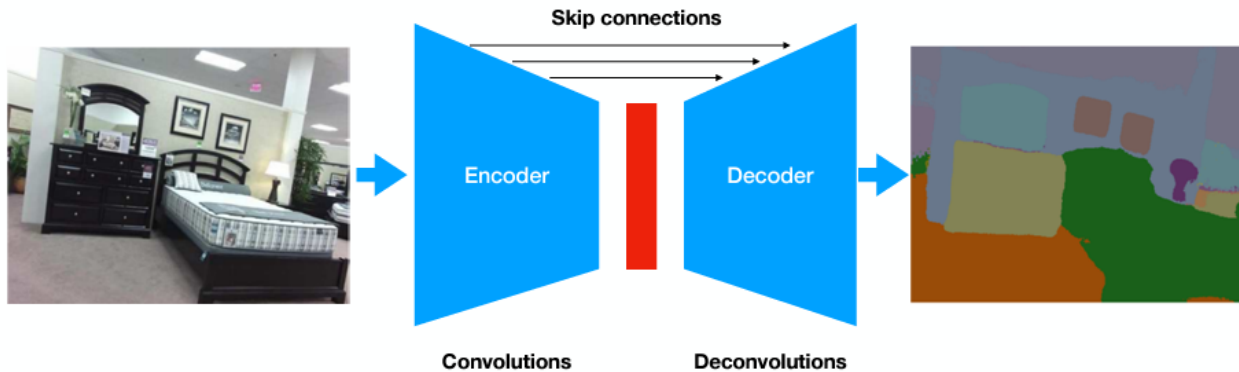
Deep Learning for Semantic Segmentation

Encoder-decoder architectures



Deep Learning for Semantic Segmentation

Encoder-decoder architectures



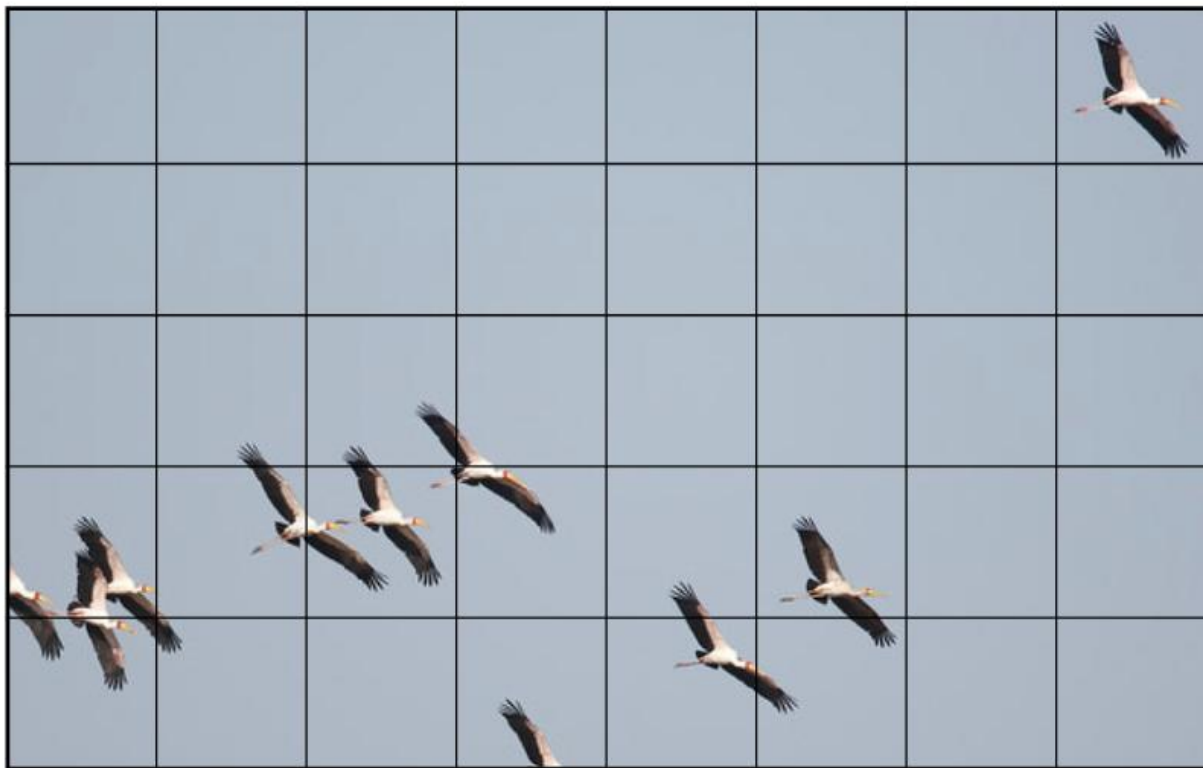
- This is essentially multi-way classification over all pixels
- Apply cross-entropy loss over all pixels

$$J(\theta) = -\frac{1}{m} \sum_i \log p(y^{(i)} | x^{(i)}) + \alpha \theta \theta^T$$

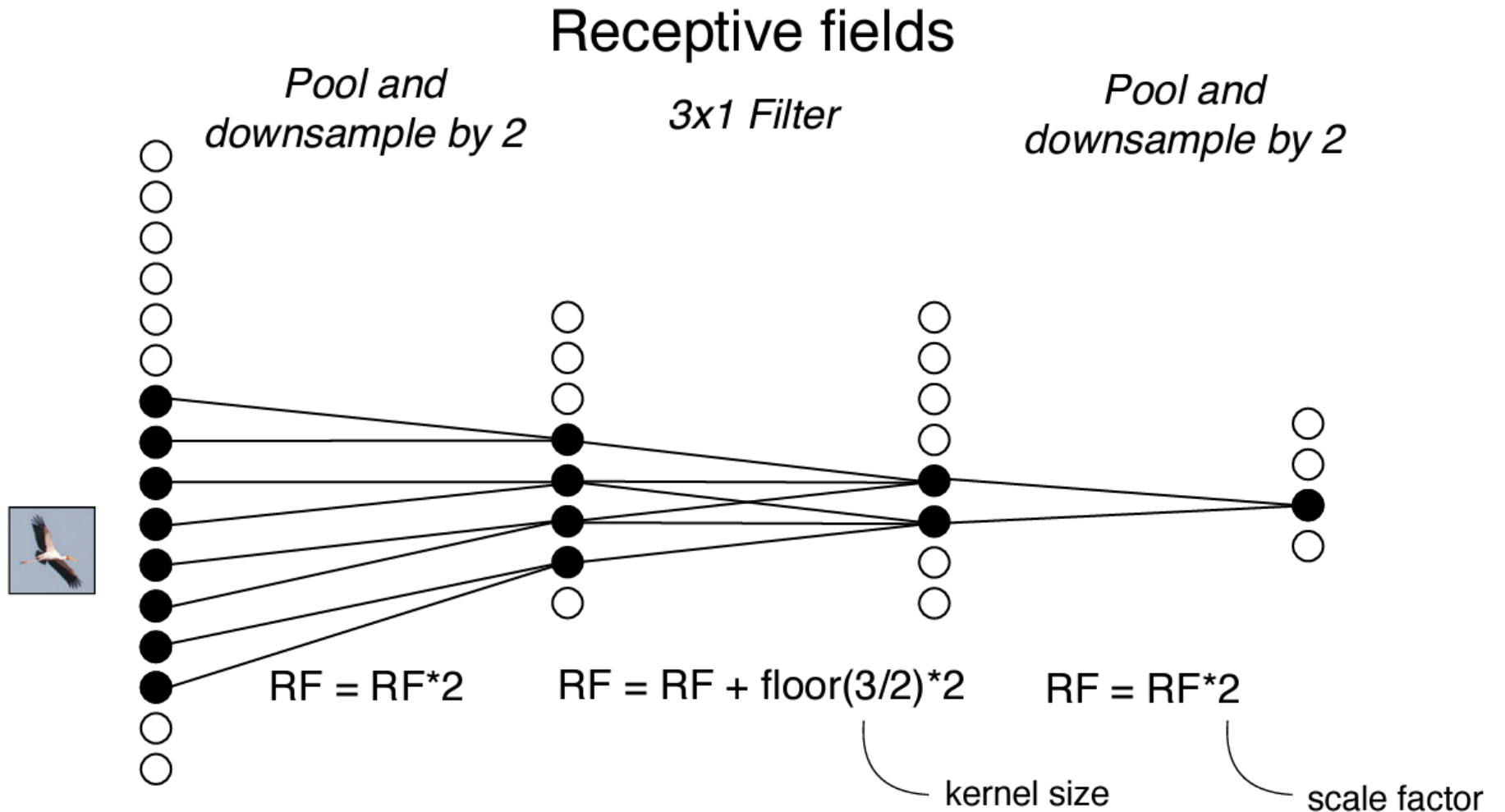
$$p(Y = y | X = x) = \frac{\exp(\theta_y \cdot x^T)}{\sum_c \exp(\theta_c \cdot x^T)}$$

Deep Learning for Semantic Segmentation -- receptive fields

Receptive fields



Deep Learning for Semantic Segmentation -- receptive fields



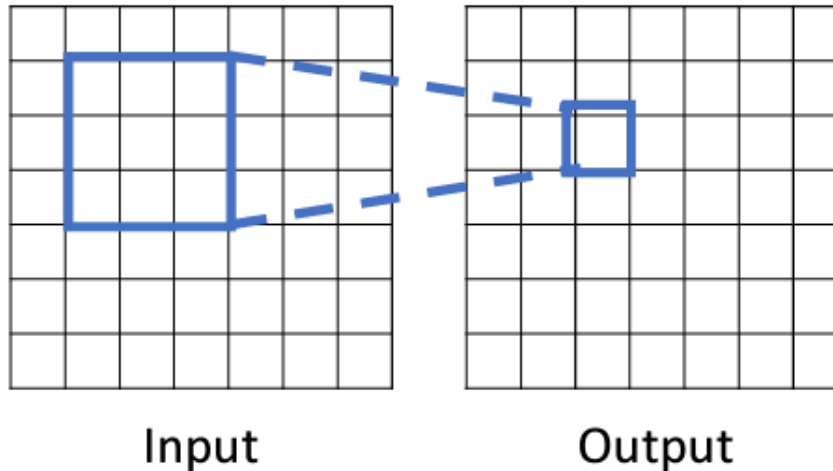
[See also: <http://fomoro.com/tools/receptive-fields/index.html>] 31

Source: Isola, Torralba, Freeman

Deep Learning for Semantic Segmentation -- receptive fields

Receptive Fields

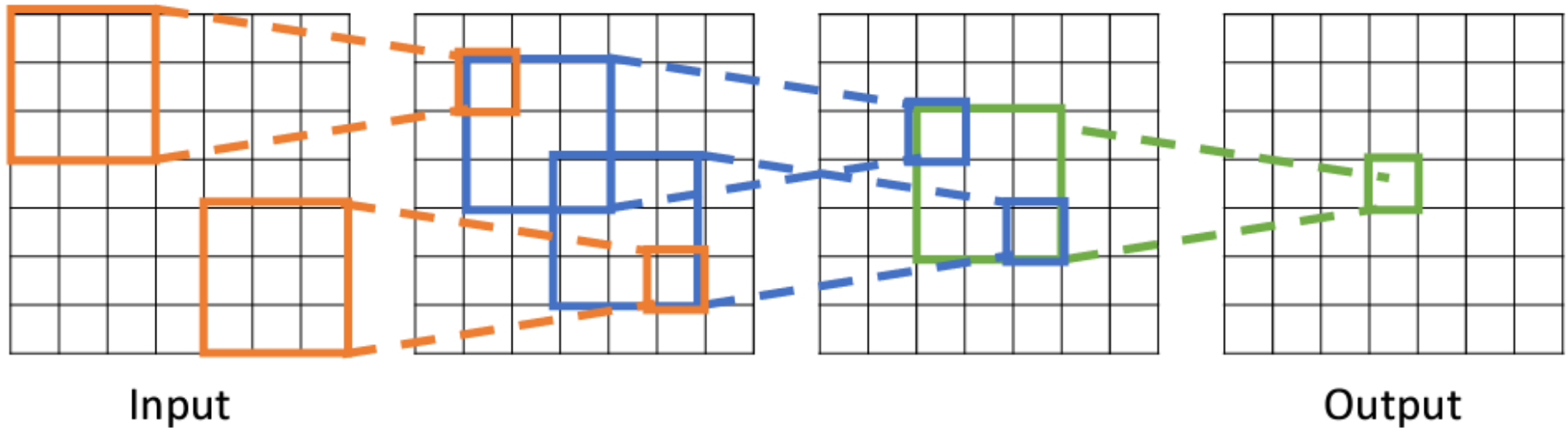
For convolution with kernel size K , each element in the output depends on a $K \times K$ **receptive field** in the input



Deep Learning for Semantic Segmentation -- receptive fields

Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$

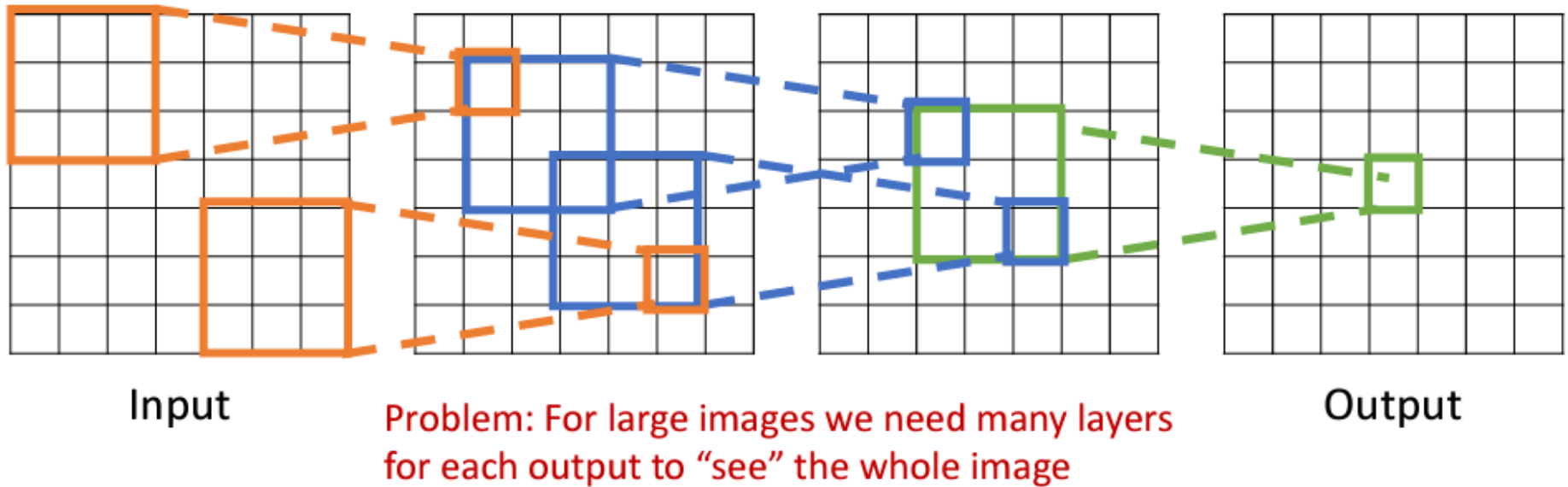


Be careful – “receptive field in the input” vs “receptive field in the previous layer”
Hopefully clear from context!

Deep Learning for Semantic Segmentation -- receptive fields

Receptive Fields

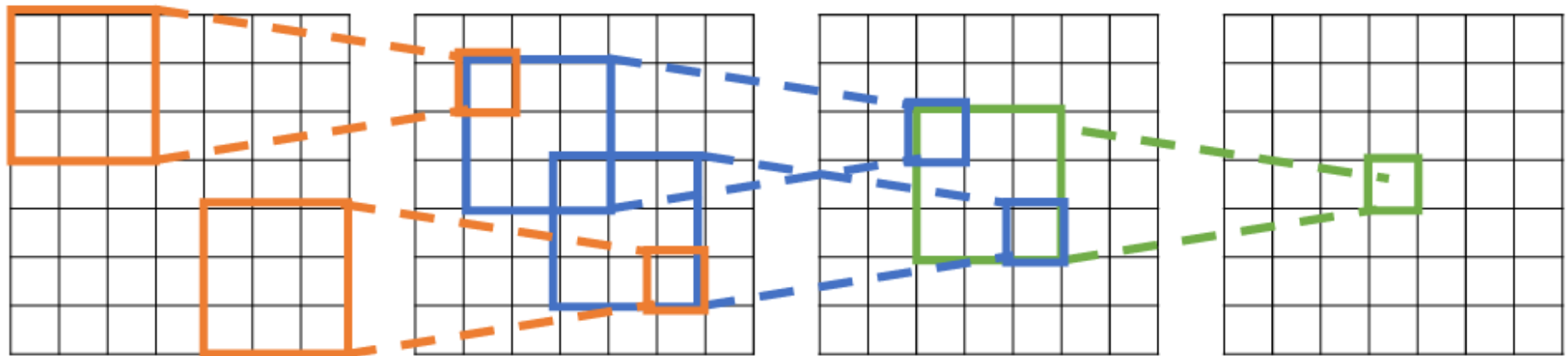
Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



Deep Learning for Semantic Segmentation -- receptive fields

Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



Input

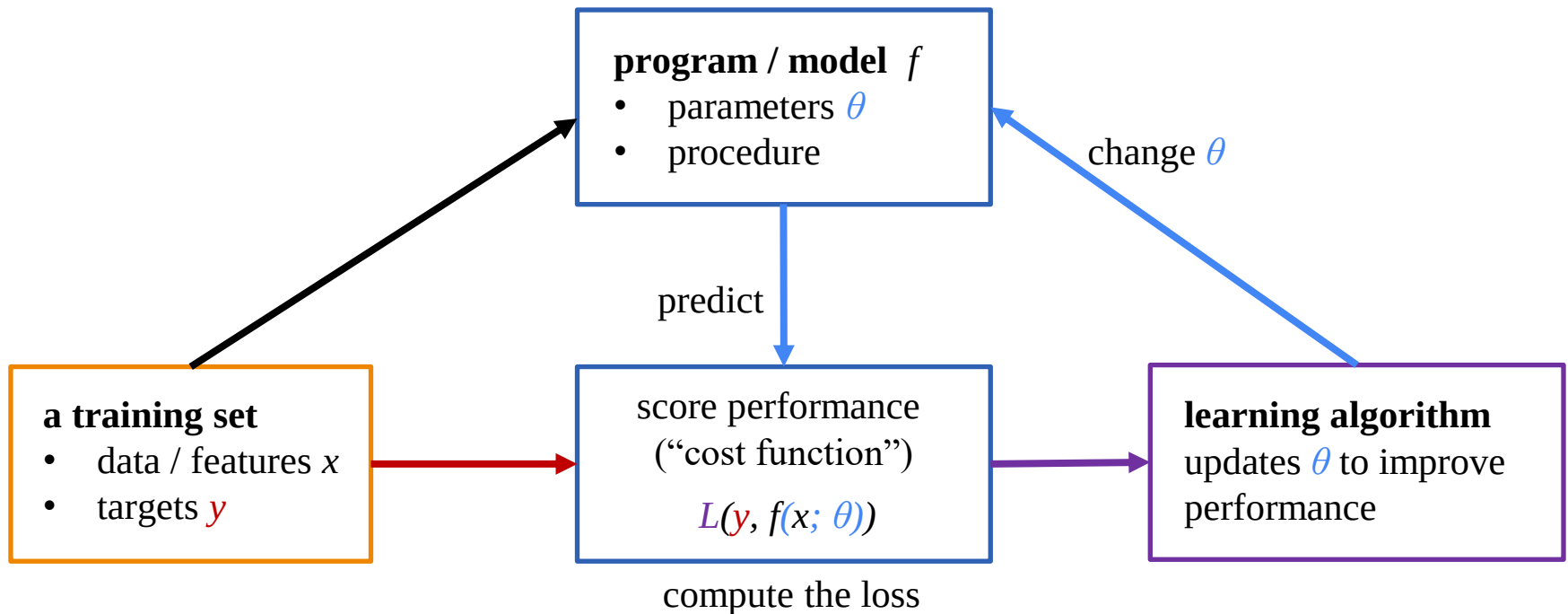
Problem: For large images we need many layers for each output to “see” the whole image

Solution: Downsample inside the network

Output

Outline

- Deep Learning for Semantic Segmentation
- **Deep Learning for Instance Segmentation**

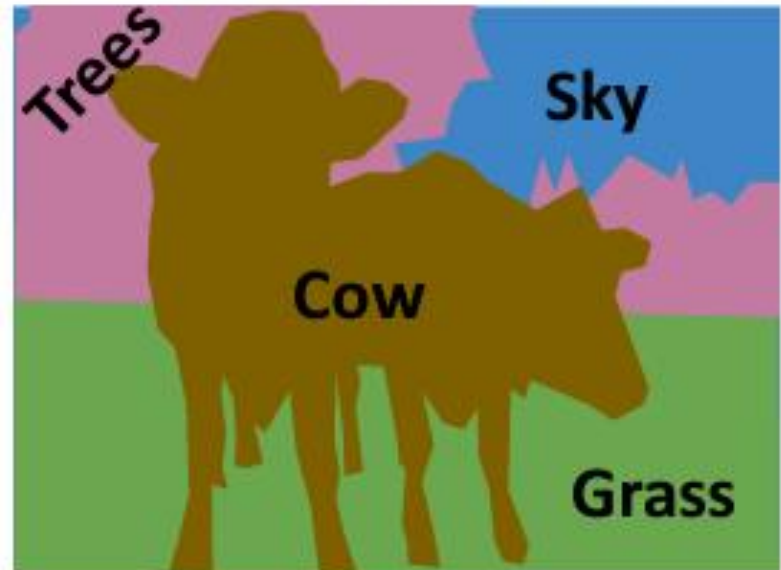


Deep Learning for Instance Segmentation

Things and Stuff

Things: Object categories that can be separated into object instances
(e.g. cats, cars, person)

Stuff: Object categories that cannot be separated into instances
(e.g. sky, grass, water, trees)



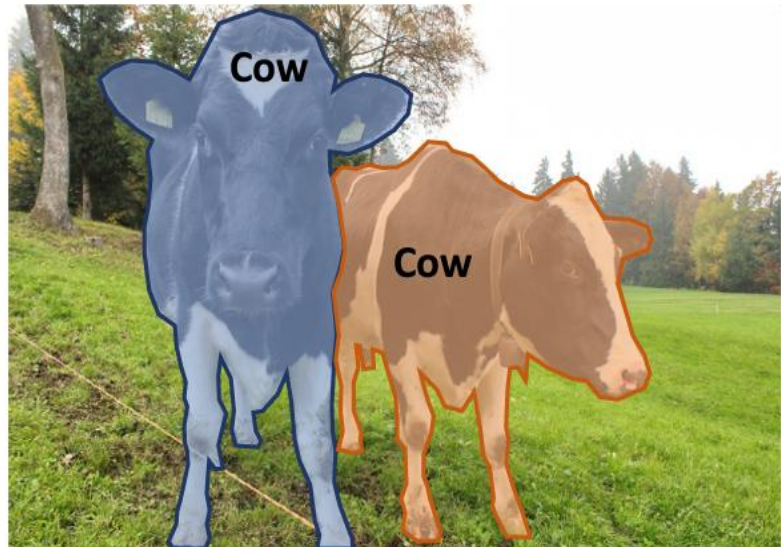
Deep Learning for Instance Segmentation

Things and Stuff

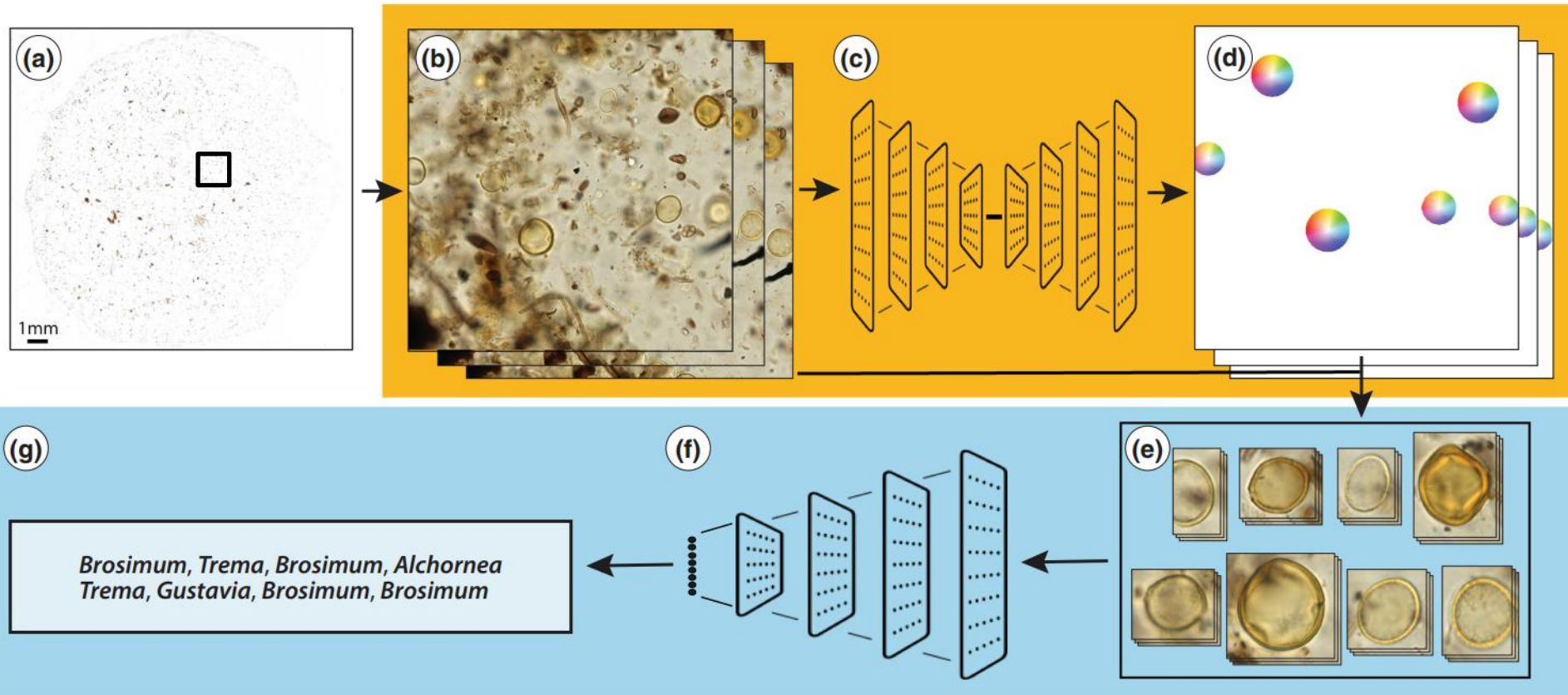
Things: Object categories that can be separated into object instances
(e.g. cats, cars, person)

Stuff: Object categories that cannot be separated into instances
(e.g. sky, grass, water, trees)

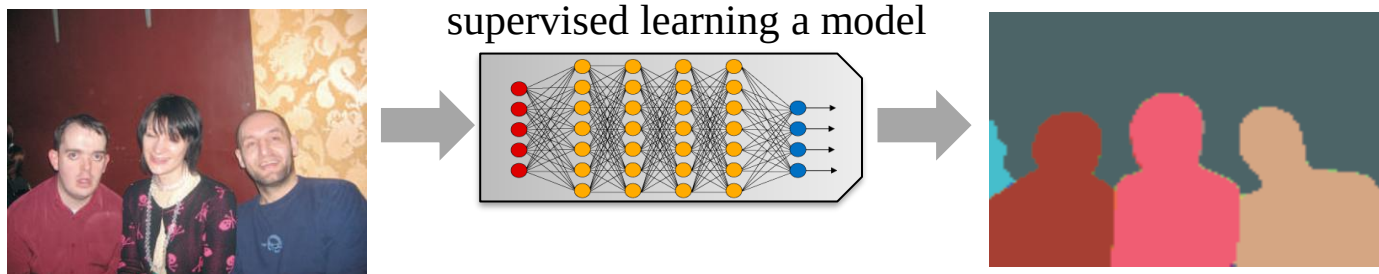
Instance Segmentation:
Detect all objects in the image, and identify the pixels that belong to each object (Only things!)



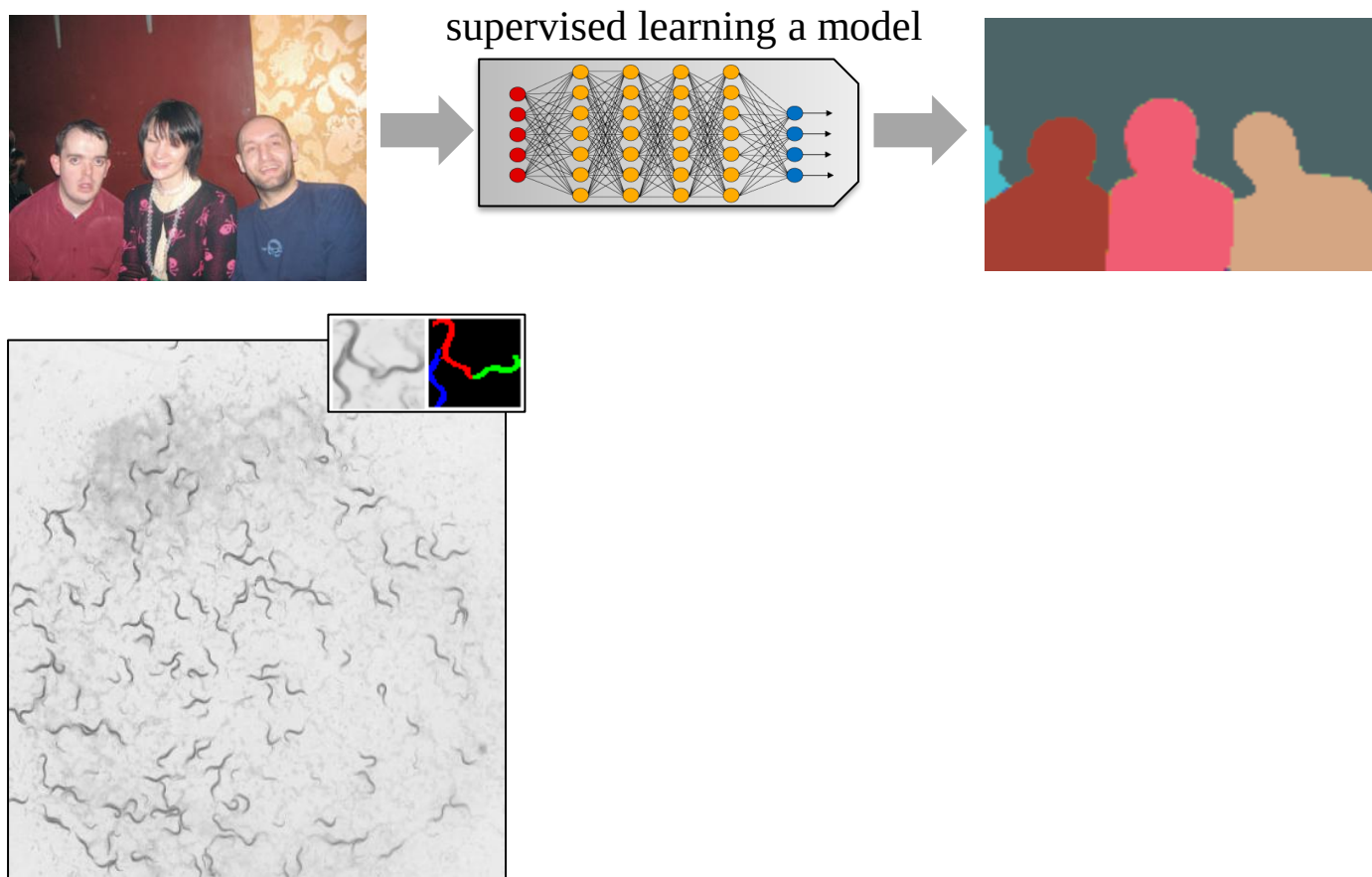
Deep Learning for Pollen Grain Segmentation/Detection



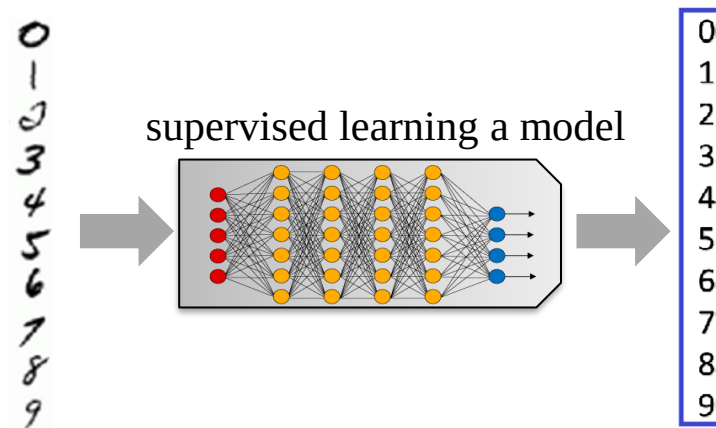
Instance Segmentation



Instance Segmentation



Recall Deep Learning for Classification



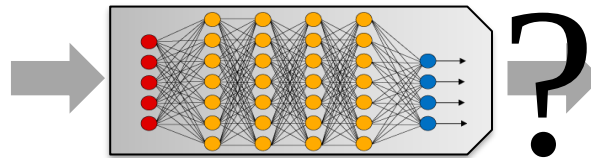
Difficulties in learning for instance segmentation

We can't do per-pixel classification

- varied number of objects
- permutable “labels”



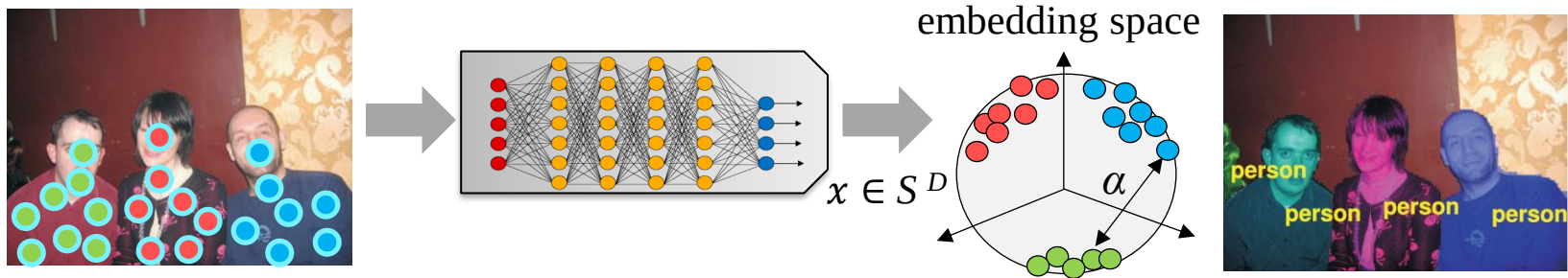
supervised learning a model



Pixel Embedding

loss design: for pixels i and j , their feature embeddings' cosine similarity s_{ij} should be:

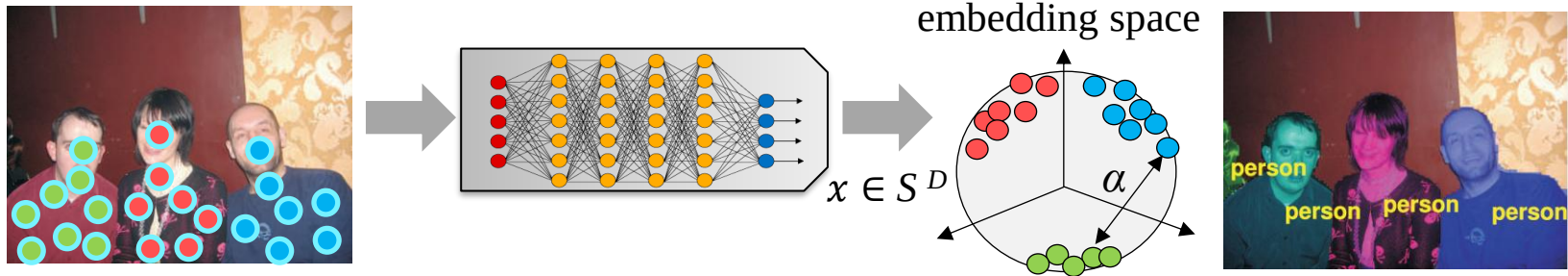
$$\left\{ \begin{array}{ll} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \\ < \alpha & \text{otherwise} \end{array} \right.$$



Pixel Embedding

loss design: for pixels i and j , their feature embeddings' cosine similarity $s_{ij} = \frac{x_i \cdot x_j}{\|x_i\|_2 \cdot \|x_j\|_2}$ should be:

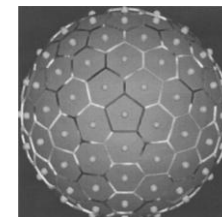
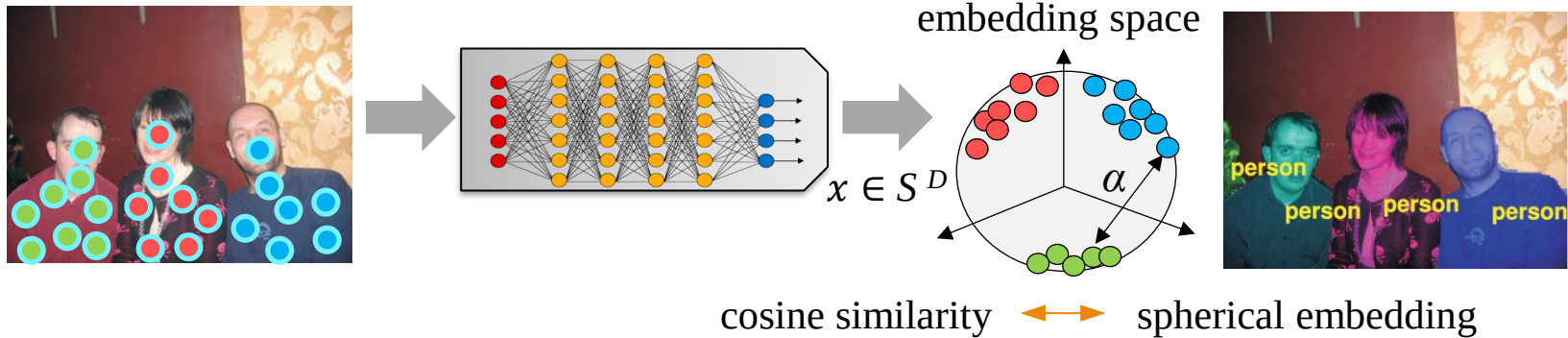
$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$



Pixel Embedding

loss design: for pixels i and j , their feature embeddings' cosine similarity $s_{ij} = \frac{x_i \cdot x_j}{\|x_i\|_2 \cdot \|x_j\|_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \\ & \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \\ & \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$



Distributing Many Points on a Sphere

E. B. SAFF AND A. B. J. KUIJLAARS

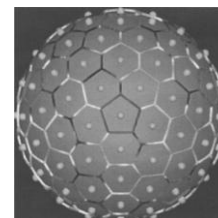
The problem of distributing a large number of points uniformly over the surface of a sphere has not only inspired mathematical researchers, it has attracted the attention of biologists, chemists, and physicists working in such fields as viral morphology, crystallography, molecular structure, and electrostatics. In two dimensions, the anal-

ogous problem is simply that of uniformly distributing points on the circumference of a disk, and equally spaced points provide an obvious answer. So we are faced with this question: What sets of points on the sphere imitate the role of the roots of unity on the unit circle?

One way such points can be generated is via optimization with respect to a suitable criterion such as “generalized energy.” Although there is a large and growing literature concerning such optimal spherical configurations of N points when N is “small,” here we shall focus on this question from an asymptotic perspective ($N \rightarrow \infty$).

Coulomb potential law is a challenging problem, sometimes referred to as the dual problem for stable molecules.

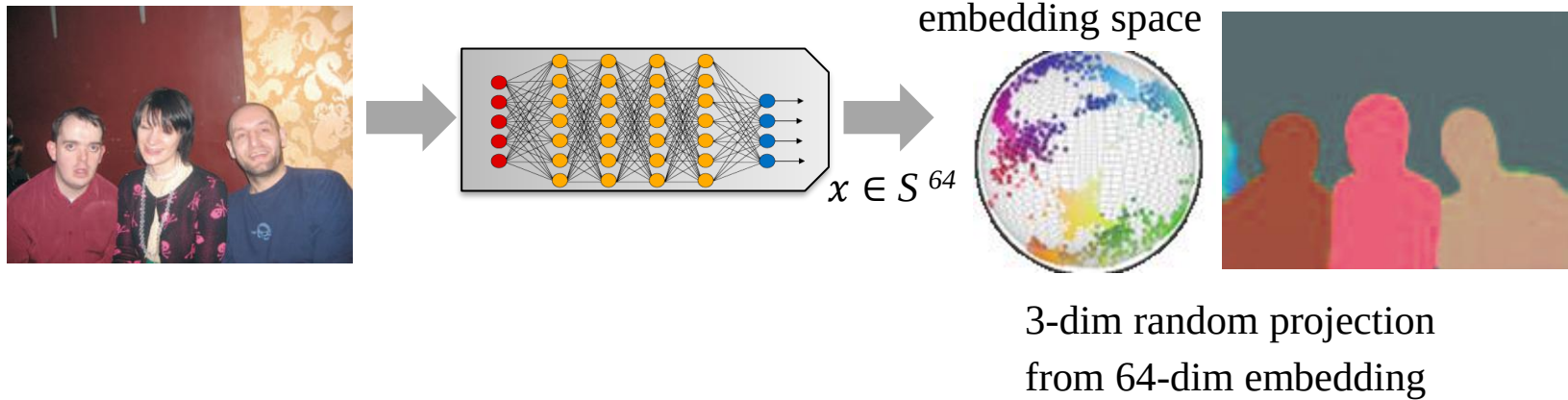
Certainly, uniformly distributing many points on the sphere has important applications to the field of computation. Indeed, quadrature formulas rely on appropriately chosen sampled data-points in order to approximate area integrals by taking averages in these points. Another example arises in the study of computational complexity, where M. Shub and S. Smale [22] encountered the problem of determining spherical points that maximize the product of their mutual distances.



Pixel Embedding

loss design: for pixels i and j , their feature embeddings' cosine similarity $s_{ij} = \frac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

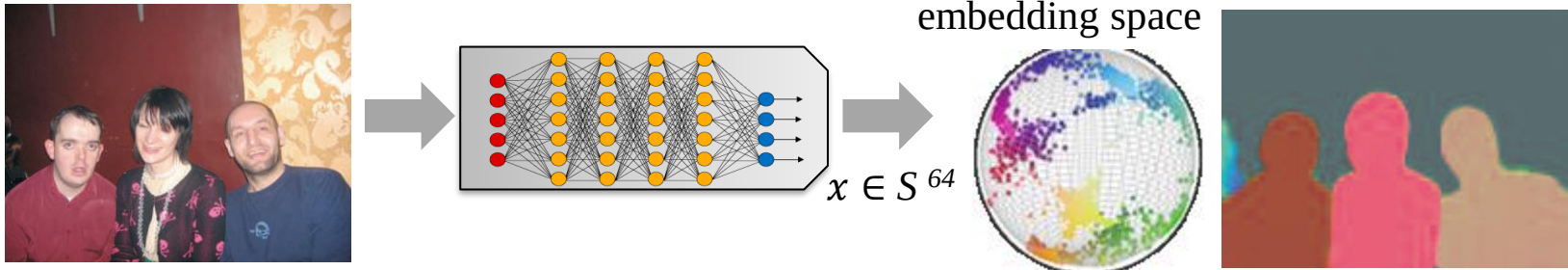
$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$



Pixel Embedding

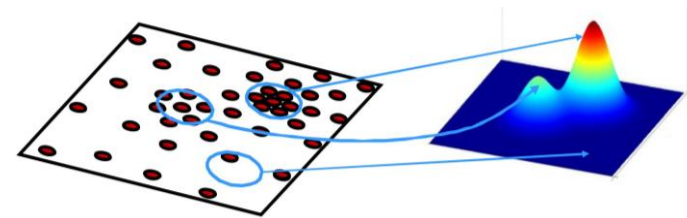
loss design: for pixels i and j , their feature embeddings' cosine similarity $s_{ij} = \frac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$



Mean Shift: iteratively seeking mode / maxima of a density function

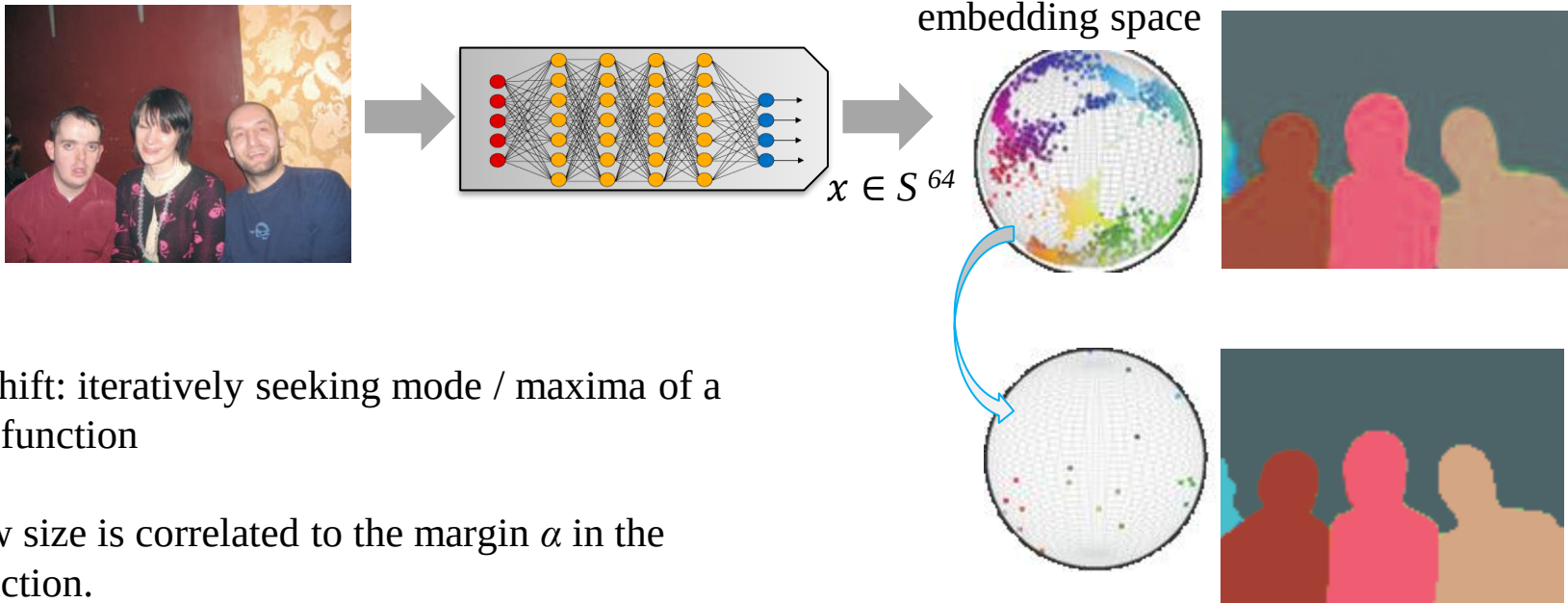
Window size is correlated to the margin α in the loss function.



Pixel Grouping

loss design: for pixels i and j , their feature embeddings' cosine similarity $s_{ij} = \frac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$



Mean Shift: iteratively seeking mode / maxima of a density function

Window size is correlated to the margin α in the loss function.

Pixel Grouping

loss design: for pixels i and j , their feature embeddings' cosine similarity $s_{ij} = \frac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

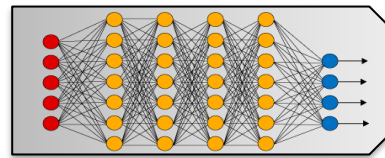
$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$

$$\mathbf{S} = \mathbf{X}^T \mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - \mathbf{1})), \mathbf{d} = \mathbf{G}^T \mathbf{1}, \mathbf{q} = \mathbf{d}^{-1}$$

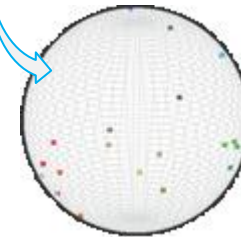
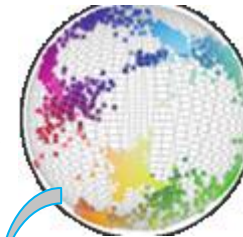
$$\mathbf{P} = \mathbf{G} \text{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1 - \eta) \mathbf{I} + \eta \mathbf{G} \text{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X} \mathbf{P}$$



$x \in S^{64}$

embedding space



Mean Shift: iteratively seeking mode / maxima of a density function

Window size is correlated to the margin α in the loss function.

Von Mises Fisher Mean Shift

L2 normalization in the loop

Algorithm 1 : vMF Mean Shift

[Normalization] $\|x_i\| = 1, \forall i.$ **for** $i = 1$ **to** N **do****[Init]** Start from the i -th sample: $y_i^{(0)} = x_i.$ **[Update]** Update the target points (vectors) until convergence by the following formula:

$$y_i^{(t+1)} \leftarrow \frac{\sum_j^N x_j g(x_j' y_i^{(t)}; \kappa)}{\|\sum_j^N x_j g(x_j' y_i^{(t)}; \kappa)\|}. \quad (6)$$

end for**[Postprocessing]** Merge close convergent points $(y_i^{(\infty)})' y_j^{(\infty)} > 1 - \epsilon$ into the same cluster.

It's guaranteed to converge without gradient vanishing or exploding.

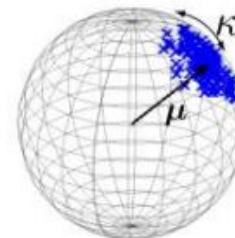
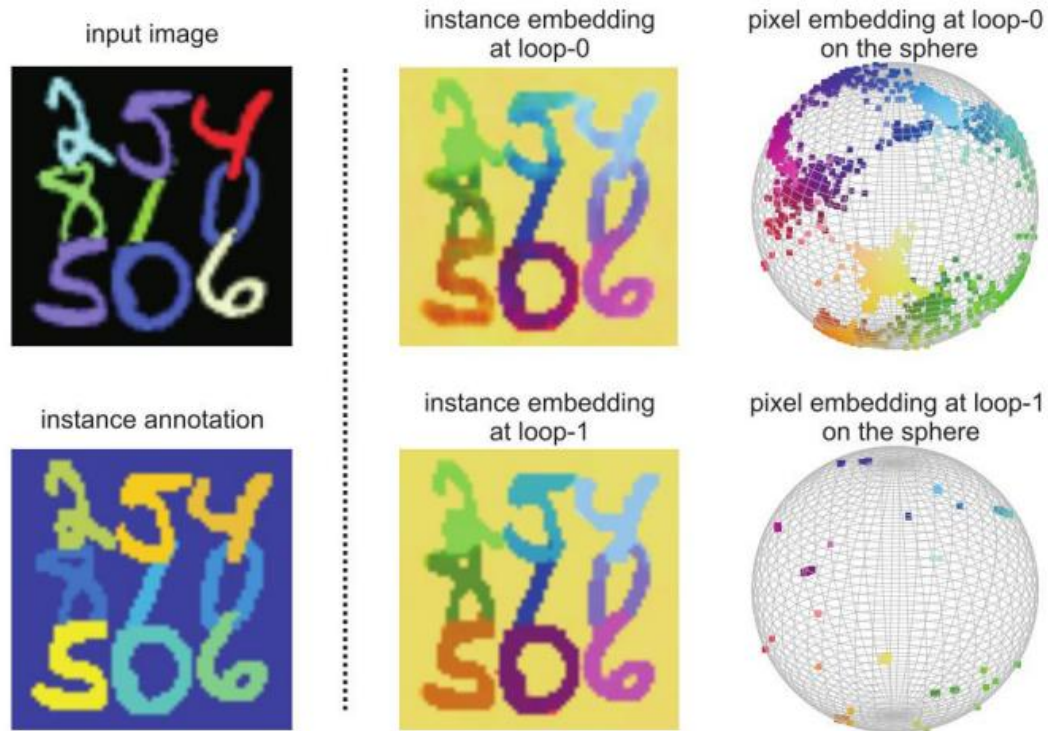


Figure 1. von Mises-Fisher distribution on a three-dimensional sphere.

Learning with Pixel Grouping

running the von-Mises Fisher mean shift offline



Demonstration. After 10 iterations of mean-shift

Learning with Pixel Grouping

$$\mathbf{S} = \mathbf{X}^T \mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - \mathbf{I})), \quad \mathbf{d} = \mathbf{G}^T \mathbf{1}, \quad \mathbf{q} = \mathbf{d}^{-1}$$

$$\mathbf{P} = \mathbf{G} \text{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1 - \eta)\mathbf{I} + \eta \mathbf{G} \text{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X} \mathbf{P}$$

Learning with Pixel Grouping

mean shift grouping in the loop

$$\mathbf{S} = \mathbf{X}^T \mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - \mathbf{I})), \mathbf{d} = \mathbf{G}^T \mathbf{1}, \mathbf{q} = \mathbf{d}^{-1}$$

$$\mathbf{P} = \mathbf{G} \text{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1 - \eta)\mathbf{I} + \eta \mathbf{G} \text{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X} \mathbf{P}$$

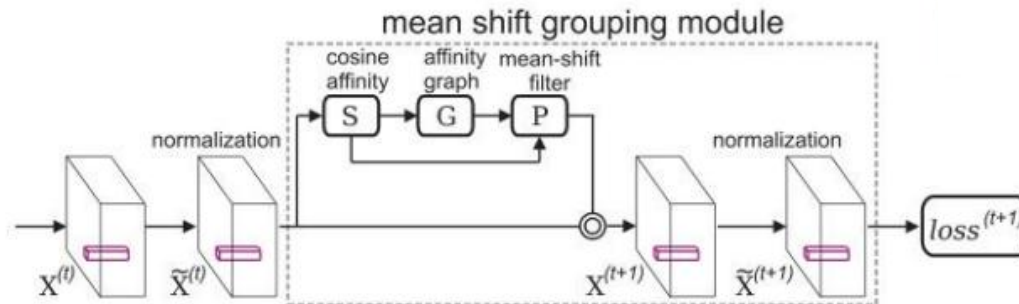
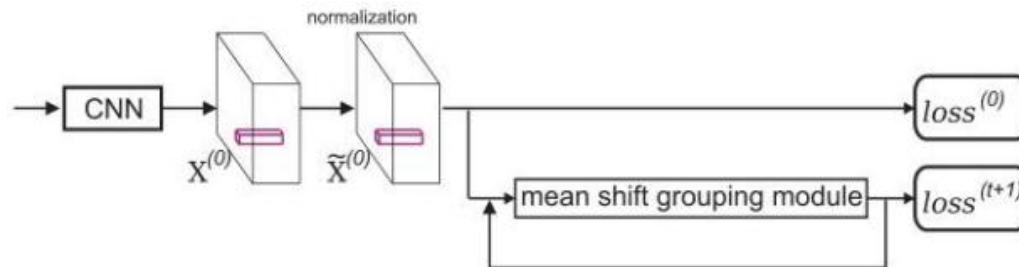


Figure 2: Mean shift grouping module.



Learning with Pixel Grouping

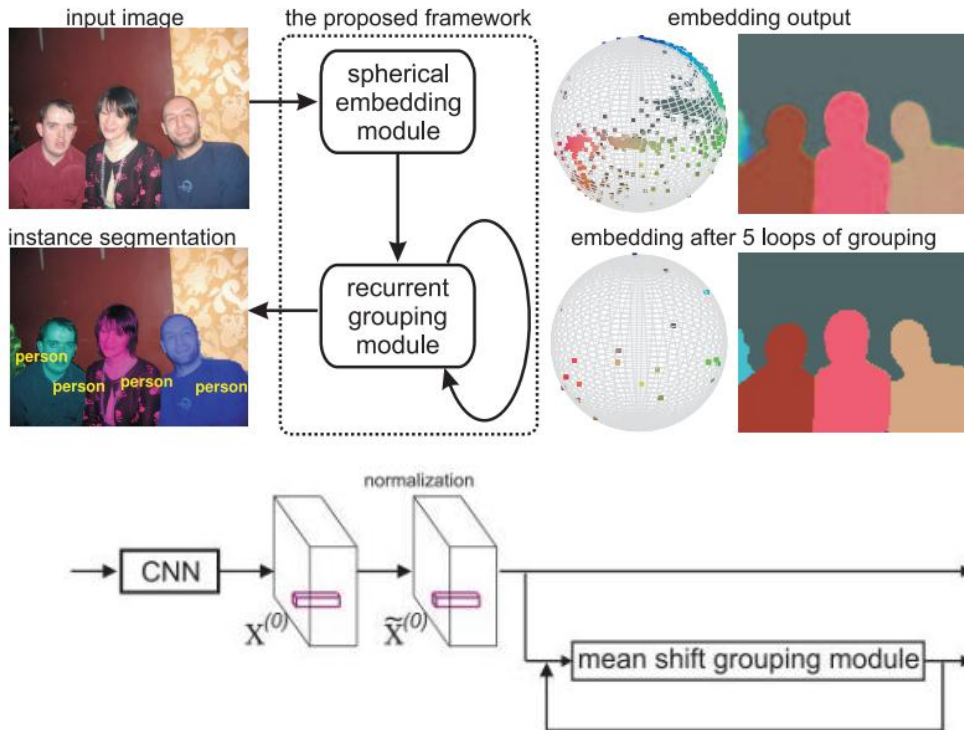
$$\mathbf{S} = \mathbf{X}^T \mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - \mathbf{I})), \mathbf{d} = \mathbf{G}^T \mathbf{1}, \mathbf{q} = \mathbf{d}^{-1}$$

$$\mathbf{P} = \mathbf{G} \text{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1 - \eta)\mathbf{I} + \eta \mathbf{G} \text{diag}(\mathbf{q})$$

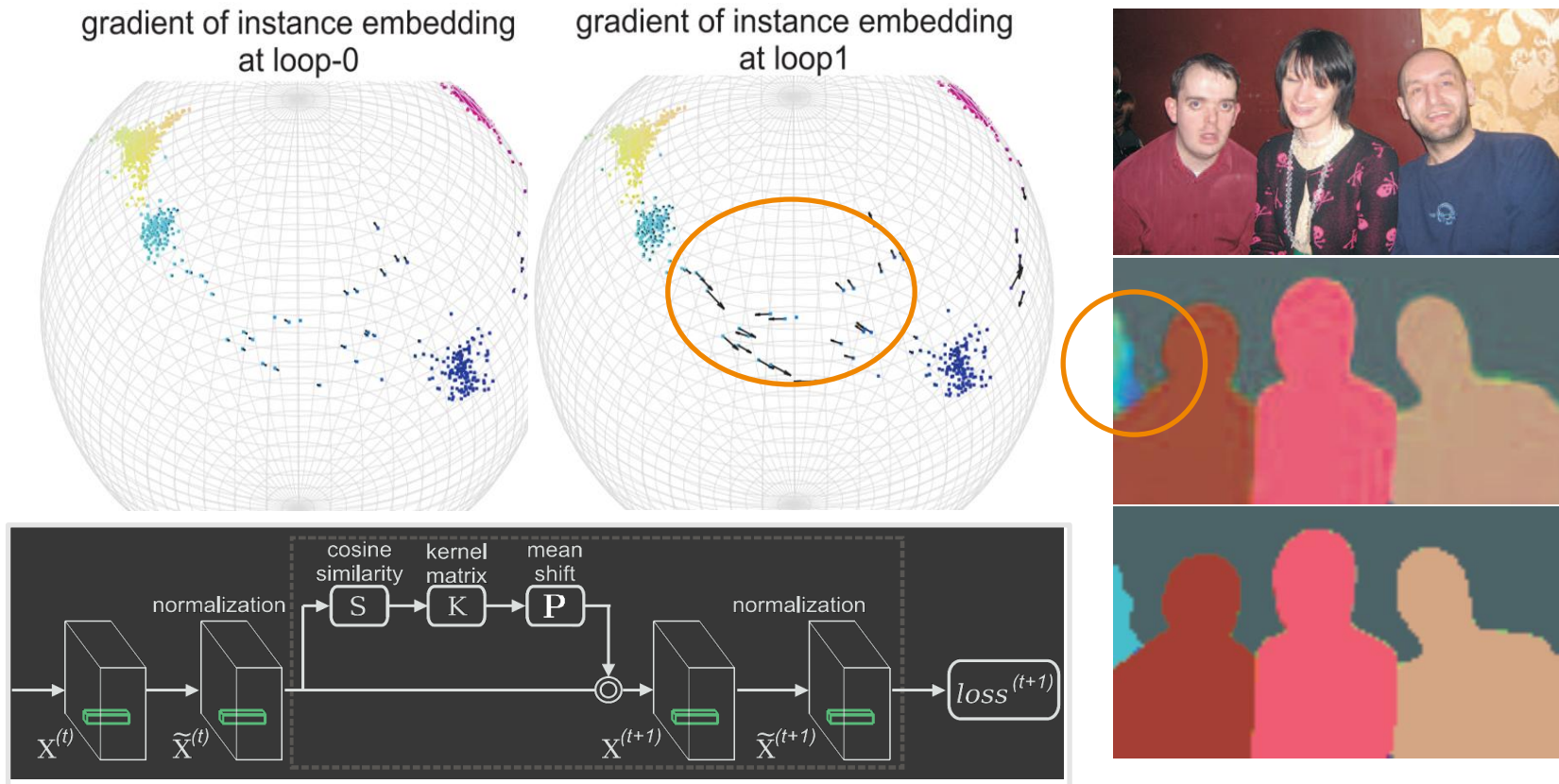
$$\mathbf{Y} = \mathbf{X} \mathbf{P}$$

mean shift grouping in the loop



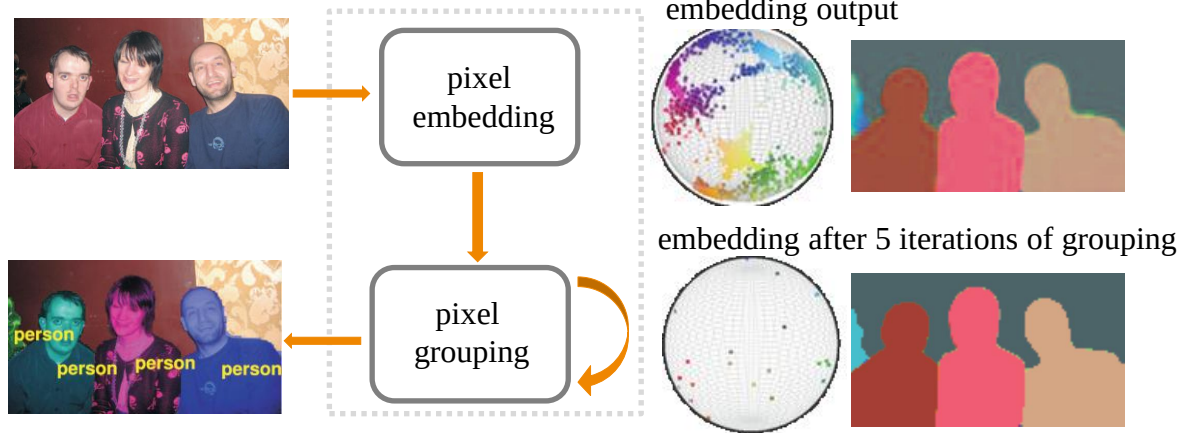
Learning with Pixel Grouping

Pixel grouping corrects embedding errors that won't be resolved in the subsequent clustering stage.



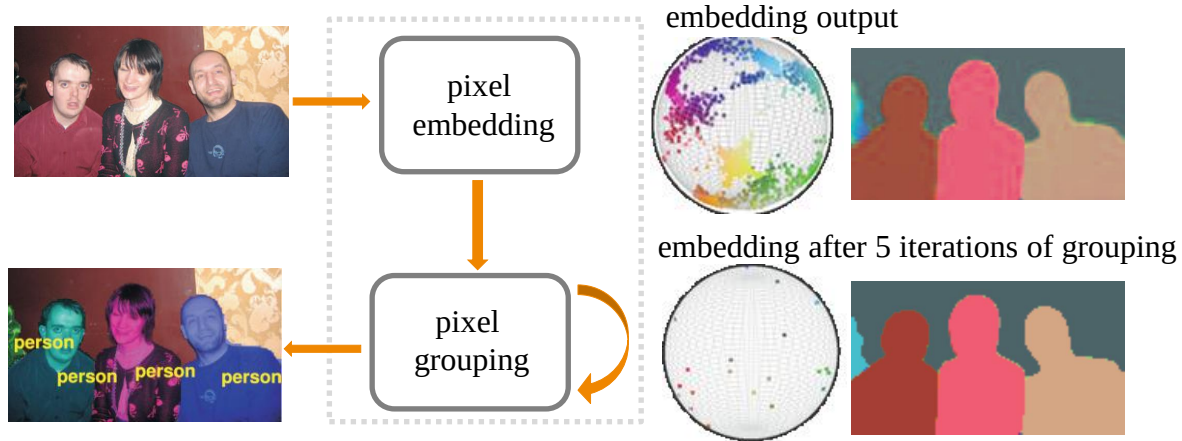
Pixel Grouping

Ours: pixel embedding and grouping

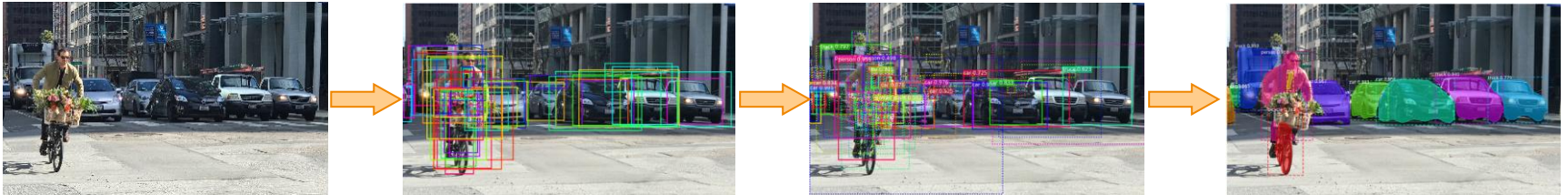


Pixel Grouping vs. Prior Art

Ours: pixel embedding and grouping

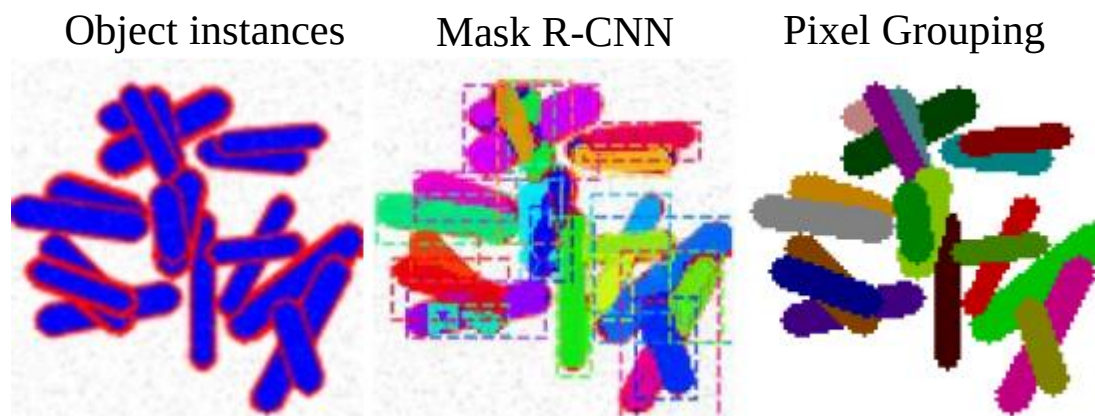


Others, e.g., Mask R-CNN



Pixel Grouping vs. Prior Art

Pixel Grouping significantly outperforms Mask R-CNN for overlapping objects.



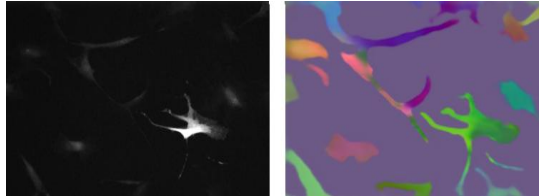
Sofiiuk et al. ICCV 2019

Pixel Grouping for More Applications

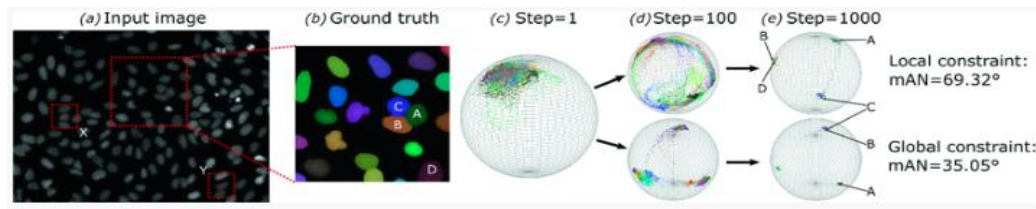
Kulikov & Lempisky CVPR 2020



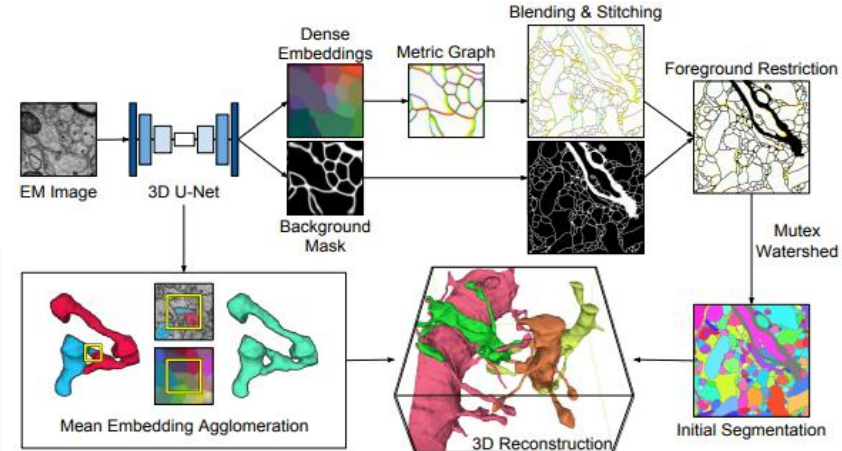
Payer et al. Medical Image Analysis 2019



Chen et al. MICCAI 2019



Lee et al. IEEE Trans. Medical Imaging, 2021

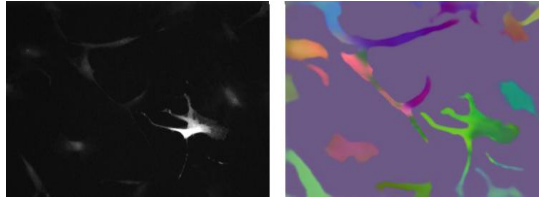


Pixel Grouping for More Applications

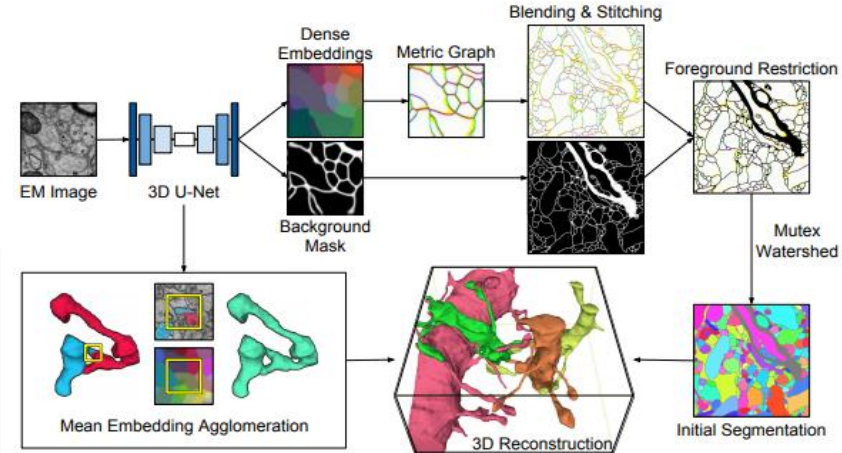
Kulikov & Lempisky CVPR 2020



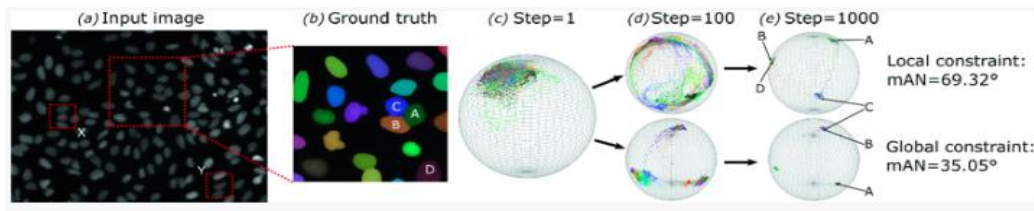
Payer et al. Medical Image Analysis 2019



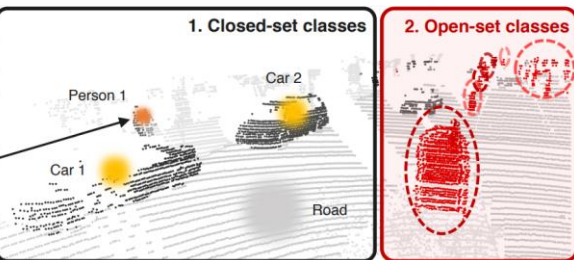
Lee et al. IEEE Trans. Medical Imaging, 2021



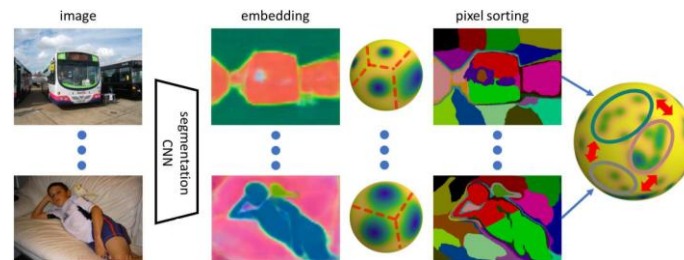
Chen et al. MICCAI 2019



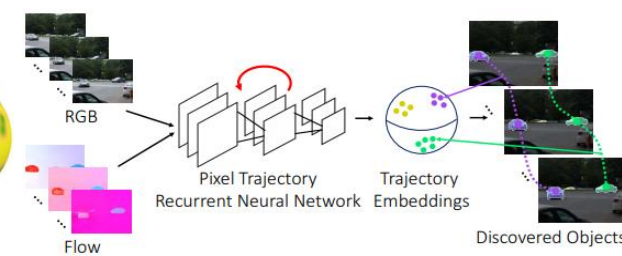
Wong et al. CoRL 2019



Hwang et al. ICCV 2019

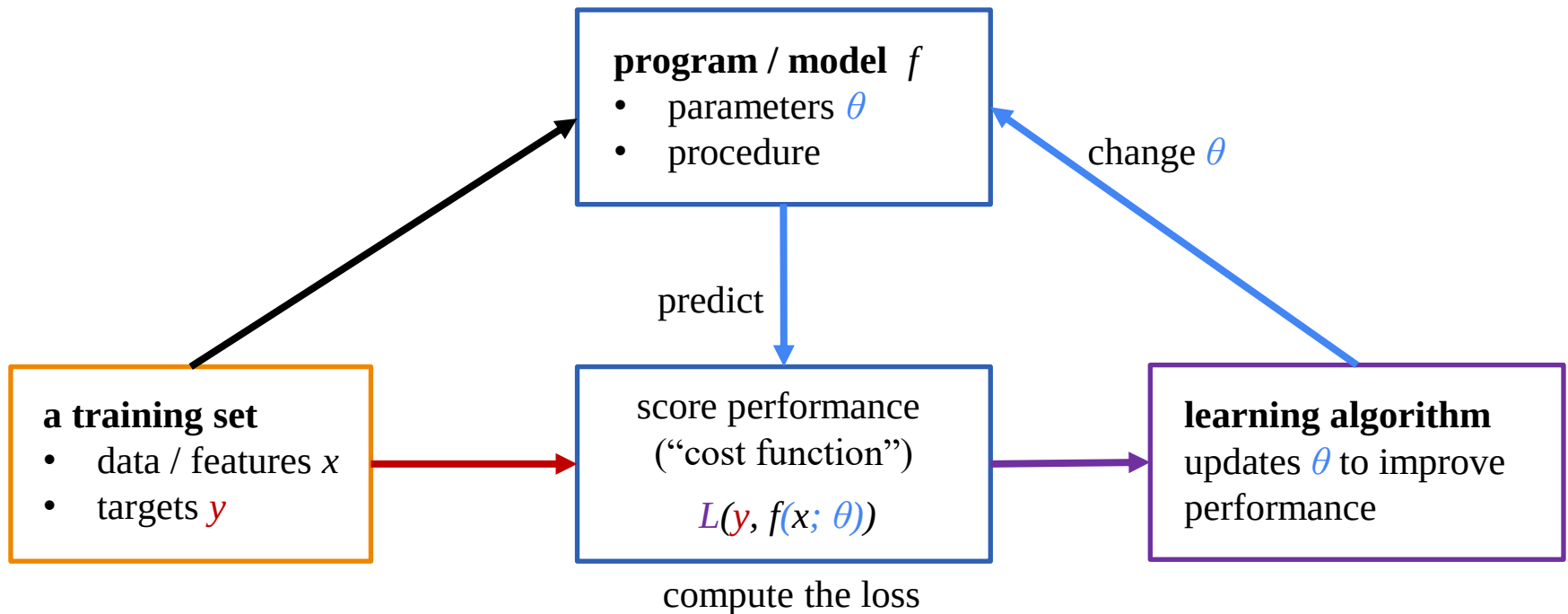


Xie et al. CVPR 2019



Outline

- Deep Learning for Semantic Segmentation
- **Deep Learning for Instance Segmentation**



Hands-on exercise

<https://github.com/aimerykong/OIST-mini-course-CVML>

Today's coding exercise:

- Train a deep neural network for pollen detection
- Download the zip file and upload to your gdrive
- Download and upload the jupyter notebooks “code005_.....ipynb” to colab

